The 30-Week Plan: A Zero-to-Production AI Engineer Roadmap

Part 1: The Modern AI Engineer: A 7-Month Battle Plan

Introduction: The 7-Month (30-Week) Challenge

This report details an intensive, 30-week curriculum designed to transition a dedicated individual from an absolute beginner to a job-ready Junior AI Engineer. This timeline is aggressive and assumes a "medium to fast learner" (as per the user query) who can commit to an "intense daily study routine" of 6 to 8 hours per day.

This 7-month (approximately 30-week) plan is predicated on consistent, disciplined effort. It is a "grind" , but one that is demonstrably achievable. While some industry experts describe a "3-6 year journey" to become a *senior* or *lead* AI engineer at a top company , other documented cases show individuals going from zero programming experience to a "Junior Gen AI Engineer" role in as little as 5.5 months.

This roadmap reconciles these timelines by focusing on a specific, modern, and in-demand role. The 7-month plan is not a path to senior-level, theoretical research; it is a pragmatic, project-based curriculum aimed squarely at a high-value *entry-level* position in the 2025 job market. This plan aligns with 6-to-12-month roadmaps that structure learning from foundational programming to advanced specialization and deployment.

The 2025 Job Title: AI Engineer vs. ML Engineer

Understanding the entire strategy of this 30-week plan requires a critical distinction between two roles that are often confused: the Machine Learning (ML) Engineer and the Artificial Intelligence (AI) Engineer.

- **The Machine Learning (ML) Engineer (Traditional):** This role is "model-focused". The ML engineer is primarily concerned with *designing, building, and scaling ML models from scratch*. Their daily work involves "data preprocessing, classical ML, predictive models, deployment pipelines," and a deep algorithmic understanding of how to train these models.
- **The AI Engineer (Modern, 2025):** This role is "system-focused". The modern AI Engineer, particularly in the post-LLM (Large Language Model) era, focuses on *using* and *integrating* existing, powerful, pre-trained models to build applications. Their world revolves around "LLMs, prompt engineering, fine-tuning, generative models," and Retrieval-Augmented Generation (RAG).

As some practitioners have noted, the job landscape has fundamentally shifted; "LLM changed the ML jobs forever". The traditional path of "AI/ML as intended before, where you needed to do a lot of modeling and fine tuning is dying fast" for many roles, being replaced by engineers who can effectively *wield* these new generative tools.

Therefore, this 30-week roadmap is strategically designed for the *AI Engineer* role. It builds a necessary foundation in classical ML (Phases 2-3) to provide context and intuition, but it then *pivots hard and fast* to the modern Generative AI stack (Phase 4). This pivot is the most efficient

path to acquiring the high-value, "real-world" skills that employers are hiring for *today*.

The 30-Week Roadmap: An Interactive Flowchart

The following flowchart provides a high-level visual of the five-phase, 30-week journey from beginner to deployed AI Engineer.

```
graph TD
    A --> B(Phase 1: Python & Math Foundations <br> Weeks 1-6);
    B --> C(Phase 2: Data Scientist's Toolkit <br> Weeks 7-12);
    C --> D(Phase 3: Classical ML & DL Foundations <br> Weeks 13-18);
    D --> E(Phase 4: The Modern GenAI Stack <br> Weeks 19-24);
    E --> F(Phase 5: MLOps & Deployment <br> Weeks 25-30);
    F --> G[Hired: Junior AI Engineer];

    subgraph Phase 1
        B1
        B2
        B3
    end

    subgraph Phase 2
        C1
        C2
        C3
        C4
    end

    subgraph Phase 3
        D1
        D2
        D3
    end

    subgraph Phase 4
        E1
        E2
        E3
    end

    subgraph Phase 5
        F1
        F2
        F3
        F4
        F5
    end

    B1-->B2-->B3;
```

```
C1-->C2-->C3-->C4;
D1-->D2-->D3;
E1-->E2-->E3;
F1-->F2-->F3-->F4-->F5;
```

Part 2: Phase 1 (Weeks 1-6): The Python & Foundational Math Vanguard

This initial phase is about forging the fundamental tools. The primary weapon of the AI Engineer is Python, and this phase is dedicated to mastering its syntax, logic, and structure. This includes the often-challenging but non-negotiable topic of Object-Oriented Programming (OOP). Concurrently, this phase builds the essential *intuition* for the core mathematics that power AI, approaching it not as an academic abstract but as the practical language of data.

Week 1: Python Basics

- **Topics:** Foundational Python syntax. This includes understanding and using variables (strings, integers, floats), the print() function for output, creating reusable blocks of code with functions, implementing logic with if/else conditions, and iterating with for and while loops.
- **Weekly Project: Simple CLI Calculator.** The learner will build a Python script that runs in the command line. The script will prompt the user for two numbers and an operator (e.g., +, -, *, /). It will then use a function and conditional logic to perform the calculation and print the result. This project solidifies mastery of variables, user input, functions, and if/else statements.

Week 2: Python Data Structures

- **Topics:** Storing collections of related data using Python's four built-in data structures: Lists, Dictionaries, Sets, and Tuples. The focus is on *when* to use each: Lists for ordered, mutable collections; Dictionaries for key-value (associative) lookups; Sets for unique, unordered items; and Tuples for immutable, fixed collections.
- **Weekly Project: Text-Based Hangman Game.** The learner will build a complete Hangman game. This project is a perfect test of data structures. It requires a List to hold the possible secret words, a Set to store the letters the user has already guessed (preventing duplicate guesses), and a List to represent the "blanks" of the word being guessed. The game loop (while) will check the game state, demonstrating practical application of all Week 1 and 2 concepts.

Week 3: Object-Oriented Programming (OOP)

- **Topics:** The principles of Object-Oriented Programming (OOP). This includes defining "blueprints" with Classes, creating "instances" called objects, understanding the __init__() constructor, bundling data as attributes, and defining behaviors with methods. The concept of inheritance, where a new class can "inherit" attributes and methods from a parent class, will also be introduced.

- **Why This is Critical:** Many beginners struggle with OOP. However, it is non-negotiable. All major AI libraries, including Pandas, NumPy, and TensorFlow, are built using the OOP paradigm. To use these tools effectively (and not just copy-paste code), one must understand how objects store data and expose methods.
- **Weekly Project: Simple Banking App.** The learner will create two classes. First, a Customer class with attributes like name and PIN. Second, an Account class that holds a Customer object as an attribute, along with a balance. The Account class will have methods like deposit(), withdraw() (which checks the balance), and check_balance(). This project replicates real-world software design, where data (attributes) and behaviors (methods) are bundled together logically.

Week 4: Algorithms & Linear Algebra Intuition

- **Topics:**
    - **Algorithms:** Implementing basic sorting algorithms like Bubble Sort or Insertion Sort. The goal is not to master sorting, but to understand algorithm *efficiency* (Big O notation) and the process of turning a logical procedure into code.
    - **Linear Algebra:** This is the *language of data*. The focus is on visual intuition, not abstract proofs. The learner will understand Vectors (a single feature list), Matrices (a whole dataset or an image), and Tensors (the general data structure).
- **Weekly Project: Visualizing Linear Algebra.** Using a plotting library (Matplotlib will be introduced early for this), the learner will plot vectors as arrows on a 2D graph. They will then define a 2x2 matrix and write a function to multiply their vectors by this matrix. By plotting the *original* and *transformed* vectors, they will visually see how matrix multiplication *transforms* (scales, rotates, shears) space. This builds the fundamental visual intuition for what PCA and SVD (Eigenvectors) are actually doing.

Week 5: Calculus & Statistics Intuition

- **Topics:**
    - **Calculus:** This is the *engine* of optimization. The key concepts are Gradients (the "slope" of an error hill), Derivatives (the "rate of change" that gives us the slope), and the Chain Rule (the mechanism for backpropagation).
    - **Statistics:** This is the *backbone* of machine learning. The focus is on descriptive statistics: Mean, Median, Mode (central tendency), Variance, Standard Deviation (data spread), and basic Probability Distributions (Normal, Binomial).
- **Weekly Project: Implementing Gradient Descent from Scratch.** The learner will define a simple mathematical function in Python, such as y = x^2. They will also write a function for its derivative, dy/dx = 2x. The script will start with a random value for x. It will then repeatedly: 1) calculate the gradient (derivative) at that x, 2) take a small step in the *opposite* direction of the gradient. The script will print the value of x and y at each step, showing how it "walks" down the curve to find the minimum y. This project is the single most important concept for understanding *all* model training.

Week 6: Working with Files & Public APIs

- **Topics:** Bridging the gap from local scripts to the outside world. This includes reading and writing data from/to text (.txt) and comma-separated-value (.csv) files. It also introduces

making GET requests to public Application Program Interfaces (APIs) using the requests library and parsing the resulting JSON data.
- **Why This is Critical:** This is the bridge to the modern AI Engineer role. AI engineers constantly pull data from various sources and interact with models via API calls.
- **Weekly Project: Public API "Weather App".** The learner will build a command-line tool that prompts the user for a city name. The script will then: 1) Make an API call to a free weather service (like OpenWeatherMap), 2) Receive a JSON response, 3) Parse the JSON to find the relevant keys (like temp and description), and 4) Print a human-readable forecast (e.g., "The weather in London is: light rain with a temperature of 15 C.").

Part 3: Phase 2 (Weeks 7-12): The Data Scientist's Toolkit

With a strong Python foundation, the learner now masters the "Data Stack." This is the day-to-day toolkit for *all* data work, built upon the libraries that form the core of the scientific Python ecosystem. This phase involves learning to manipulate massive datasets efficiently (NumPy, Pandas) , visualize patterns (Matplotlib) , and build the first predictive models (Scikit-learn).

Week 7: NumPy

- **Topics:** The core data structure of all data science: the NumPy ndarray. The learner will master vectorization (why for loops are slow), array indexing and slicing, broadcasting (how NumPy handles operations between arrays of different shapes), and high-speed mathematical operations on matrices.
- **Weekly Project: Image Manipulation with NumPy.** An image is just a 3D NumPy array (width, height, color channels). The learner will load a simple image into an ndarray. They will then use vectorized operations to: 1) Change its brightness (by adding a constant to the entire array), 2) Increase its contrast (by multiplying the array), and 3) Crop the image (using array slicing). This project directly connects the abstract concept of "matrices" to a tangible, visual output.

Week 8: Pandas (Part 1 - Cleaning)

- **Topics:** The DataFrame and Series—the most common objects for working with tabular data. This week focuses on data *cleaning*. Topics include: reading data from CSV/Excel files (pd.read_csv), finding and handling missing data (.dropna(), .fillna()), correcting data types (.astype()), and cleaning text fields using string manipulation (.str()).
- **Weekly Project: Real-World Data Cleaning.** The learner will be given a notoriously "messy" real-world dataset (e.g., a movie budget dataset , a city bike dataset , or a Star Wars survey). Their task is to perform a data-cleaning audit. They will load the data, identify all missing values, duplicates, and invalid entries (e.g., a "Yes/No" column with "Yes", "No", and "Y" entries). The final deliverable is a *new, clean* CSV file and a Jupyter Notebook documenting all the changes made.

Week 9: Pandas (Part 2 - Analysis)

- **Topics:** Exploratory Data Analysis (EDA). With clean data, the learner now focuses on *asking questions*. Topics include: filtering data with loc and iloc, grouping data with .groupby(), performing aggregations (.mean(), .sum(), .value_counts()), and merging/joining two separate DataFrames together.
- **Weekly Project: Exploratory Data Analysis (EDA) on Telecom Churn.** Using a classic "Telecom Churn" dataset , the learner will act as a data analyst. They will use Pandas to answer specific business questions: "What is the overall churn rate?" , "Which 'Total day charge' value is most common?", "Do customers with more 'vmail messages' churn less?", and "What are the average call charges for customers who churned vs. those who didn't?".

Week 10: Matplotlib & Seaborn

- **Topics:** The "grammar" of data visualization. The learner will master the standard plotting libraries. Matplotlib is used for basic, customizable plots (line plots, bar charts, histograms, scatter plots). Seaborn, which is built on Matplotlib, is used for more complex statistical visualizations like correlation heatmaps and boxplots.
- **Weekly Project: EDA Visualization Dashboard.** Building directly on the Week 9 project, the learner will use a Jupyter Notebook to create a visual report. They will take their answers from the previous week and visualize them. This includes: a histogram of "Total day charge," a bar chart of "Churn," a boxplot showing the distribution of "Total day charge" for churned vs. non-churned customers, and a correlation heatmap of all numerical features to see what is most related.

Week 11: Scikit-Learn (Part 1 - First Model)

- **Topics:** Introduction to the "Estimator" API, the simple and consistent interface used by Scikit-learn: .fit(), .predict(). This is the first week of *Supervised Learning*. The learner will split data into training and testing sets (train_test_split) and build and understand the two most fundamental models: Linear Regression (for predicting continuous values) and Logistic Regression (for predicting categories).
- **Weekly Project: House Price Prediction (Regression).** Using a "Boston Housing" or "California Housing" dataset , the learner will build their first end-to-end ML model. They will: 1) Load the data, 2) Split it into training and testing sets, 3) fit() a LinearRegression model on the training data, and 4) predict() prices on the test data. Finally, they will evaluate the model's performance by calculating the Mean Squared Error.

Week 12: Scikit-Learn (Part 2 - Evaluation)

- **Topics:** How to know if a model is *actually* good. This week focuses on classification model evaluation metrics: Accuracy, Precision, Recall, and F1-score. The learner will master the Confusion Matrix, which is the "scoreboard" for a classifier. This week also introduces the concepts of Overfitting (doing well on training data, poorly on test data) and Cross-Validation (a more robust method for evaluating performance).
- **Weekly Project: Breast Cancer Classifier (Logistic Regression).** Using the "Breast Cancer" dataset , the learner will build a LogisticRegression classifier. The project has a specific, multi-step goal: 1) Split the data , 2) Train the model, 3) Evaluate its performance using a confusion matrix and report the accuracy, precision, and recall. 4) *Critically*, they

will then re-evaluate the model using 5-fold cross-validation to get a more robust and reliable performance score. This teaches them to be skeptical of a single "accuracy" number.

Part 4: Phase 3 (Weeks 13-18): Classical ML & Deep Learning Foundations

In this phase, the learner moves beyond simple linear models to the "workhorse" algorithms of classical ML. The objective is not to become a world-expert in any single algorithm, but to build a strong *intuition* for *how* different models "think"—some use geometry (SVMs), some use probability (Naive Bayes), and some use logic (Decision Trees). This phase then culminates in the critical leap to Deep Learning, building the foundational neural network architectures (ANNs and CNNs) that power the entire modern AI field.

Week 13: Supervised Learning (Trees)

- **Topics:** Decision Trees (DTs) and their powerful "ensemble" version, Random Forests (RFs). DTs work by learning a series of simple if/else rules from the data. Random Forests improve upon this by building *many* "weak" trees on random subsets of data and having them "vote," which dramatically reduces overfitting.
- **Weekly Project: Spam Classifier (Part 1).** Using a "SMS Spam" dataset , the learner will build a text classifier. This requires a new step: preprocessing the text. The learner will use Scikit-learn's CountVectorizer or TfidfVectorizer to convert text messages into a numerical matrix (a bag-of-words). They will then train a RandomForestClassifier on this matrix and compare its performance to the Logistic Regression model from Week 12.

Week 14: Supervised Learning (Other)

- **Topics:** Expanding the toolkit with other powerful classifiers. This includes:
    - **Support Vector Machines (SVMs):** A geometric model that finds the "widest possible street" (hyperplane) to separate data points.
    - **K-Nearest Neighbors (KNN):** A simple, instance-based model that classifies a new point by a "majority vote" of its k closest neighbors.
    - **Naive Bayes:** A probabilistic classifier that uses Bayes' Theorem to calculate the probability of a class given a set of features.
- **Weekly Project: Spam Classifier (Part 2).** This project teaches the real-world task of model selection. The learner will re-build the spam classifier from Week 13, but this time using a MultinomialNB (Naive Bayes) model and a LinearSVC (SVM) model. They will then create a comparison table (in their notebook) showing the accuracy, precision, and recall for all three models (Random Forest, Naive Bayes, SVM). This demonstrates how to empirically select the best model for a given task.

Week 15: Unsupervised Learning (Clustering)

- **Topics:** Shifting from *Supervised* (labeled data) to *Unsupervised* (unlabeled data) learning. The goal of clustering is to find "natural groups" in the data. The learner will master the K-Means Clustering algorithm, the most popular and intuitive clustering

method. This includes learning how to use the "Elbow Method" to find the optimal number of clusters (k).
- **Weekly Project: Customer Segmentation.** Using the "Mall Customer" dataset, which contains features like age, annual income, and spending score , the learner will use K-Means to cluster customers. They will first use the "Elbow Method" to plot the "inertia" (within-cluster sum-of-squares) for k=1 through k=10 to justify their choice of k (e.g., k=5). The output will be a DataFrame with a new "cluster" column, identifying distinct segments like "high income, low spend" or "low income, high spend."

Week 16: Unsupervised Learning (Dimensionality Reduction)

- **Topics:** The "curse of dimensionality"—why models often fail when given too many features. The learner will master Principal Component Analysis (PCA) as a technique for dimensionality reduction. This connects directly back to Phase 1: PCA works by finding the Eigenvectors (the "principal components") of the data's covariance matrix, which represent the "directions of highest variance".
- **Weekly Project: Visualizing Customer Segments.** This project provides a powerful "Aha!" moment. The customer data from Week 15 is 3D+ (age, income, score). This makes it hard to visualize. The learner will take that data, use PCA to reduce its dimensions from 3D to 2D , and *then* create a 2D scatter plot. The x-axis will be "Principal Component 1" and the y-axis will be "Principal Component 2". They will color the dots on this plot based on the cluster ID from Week 15. For the first time, they will be able to *see* the distinct clusters their algorithm found.

Week 17: Deep Learning (Intro to PyTorch & ANNs)

- **Topics:** The major leap into Deep Learning.
    - **Frameworks:** A discussion of TensorFlow vs. PyTorch. This roadmap selects PyTorch, as it is often praised for its "Pythonic" nature, intuitive syntax, and dominance in research, making it easier for beginners to grasp.
    - **Core Concepts:** The PyTorch nn.Module (the "blueprint" for all models). Artificial Neural Networks (ANNs), also called Multi-Layer Perceptrons (MLPs). The learner will understand the basic components: Layers, Neurons, Weights, Biases, and Activation Functions (like ReLU and Sigmoid).
- **Weekly Project: Iris Classifier with an ANN.** The learner will build their first neural network. They will take the classic "Iris" dataset (which they might have used with Scikit-learn), define a simple ANN (e.g., 4 input features, 1 hidden layer of 10 neurons, 3 output classes), and write the PyTorch training loop (forward pass, loss calculation, backpropagation, optimizer step). This replaces a Scikit-learn model with a custom-built neural network.

Week 18: Deep Learning (Convolutional Neural Networks - CNNs)

- **Topics:** Why ANNs (which are "fully connected") fail on image data. Introduction to the core concepts of Convolutional Neural Networks (CNNs), the architecture that powers most computer vision. The learner will master the two key layer types:
    - **Convolution Layers:** These act as "feature detectors" (filters) that scan for edges, shapes, and textures.

    - **Pooling Layers:** These "downsample" the image, reducing its size while keeping the most important information.
- **Weekly Project: Handwritten Digit Recognition (MNIST).** This is the foundational "Hello, World!" project of deep learning. The learner will build a simple CNN in PyTorch to classify the "MNIST" dataset of handwritten digits (0-9). They will define a model with 2-3 convolution/pooling blocks followed by a fully connected (ANN) "head." They will train this model and watch it achieve >98% accuracy on data it has never seen. **This trained model will be a key asset saved for the deployment phase.**

Part 5: Phase 4 (Weeks 19-24): The Modern Generative AI Stack

This phase is the *pivot*. Having built a solid foundation in classical ML and the *architecture* of deep learning, the learner now "graduates" from building small models from scratch to *using* massive, state-of-the-art, pre-trained models. This is the core of the 2025 AI Engineer role. This phase is entirely dedicated to the "Generative" stack: Hugging Face (the model hub) , Embeddings (the language of meaning) , Vector Databases (the "memory" for AI) , and Retrieval-Augmented Generation (RAG) (the system that ties it all together).

Week 19: Transformers & Hugging Face (The Easy Way)

- **Topics:** A high-level introduction to the Transformer architecture (which powers models like GPT and BERT). An introduction to the Hugging Face (HF) ecosystem, which is the "GitHub for AI models". The focus is on the simple, high-level pipeline() function, which abstracts all the complexity.
- **Weekly Project: One-Line AI Tools.** The learner will build a simple Python script that demonstrates the power of the HF pipeline(). In just a few lines of code each, they will implement:
    1. A Sentiment Analysis tool.
    2. A Text Summarization tool.
    3. An Image Captioning tool. This project's goal is to produce a "wow" moment, demonstrating the immense power available "off-the-shelf".

Week 20: Hugging Face (The "Real" Way)

- **Topics:** Going "under the hood" of the pipeline(). The learner will be introduced to the two key components:
    1. AutoTokenizer: The object that converts human-readable text into numerical "tokens" that a model understands.
    2. AutoModel: The actual pre-trained model (e.g., BERT) loaded from the hub. The learner will understand what "tokens" and "attention masks" are and why they are necessary.
- **Weekly Project: Re-Build the Sentiment Analyzer.** The learner will replicate the Week 19 project, but *without* using the pipeline() function. They must manually: 1) Load the AutoTokenizer and AutoModel for a sentiment task, 2) Tokenize a sentence (convert it to input_ids and attention_mask), 3) Pass these tensors to the model, 4) Receive the output "logits," and 5) Apply a softmax function to interpret the logits as a human-readable

prediction. This is a *critical* step in de-mystifying what these models actually do.

Week 21: Embeddings & Semantic Search

- **Topics:** What *are* embeddings? They are the *output* of models (like the ones from Week 20). An embedding is a high-dimensional vector (connecting back to Linear Algebra, Week 4) that numerically represents the *meaning* or *semantic content* of a piece of data (text, image, etc.). The learner will use a specific HF sentence-transformer model (e.g., all-MiniLM-L6-v2) to turn text into vectors. They will also learn to use Cosine Similarity (the mathematical formula) to measure the "closeness" of two vectors.
- **Weekly Project: Semantic Search Engine (Local).** The learner will: 1) Create a list of 50 example sentences, 2) Use a sentence-transformer model to embed all 50 sentences, storing them in a list (or NumPy array), 3) Write a function that takes a new "query" sentence, 4) Embeds the query, and 5) Uses a for loop to calculate the cosine similarity between the query vector and all 50 stored vectors. The function will then return the top 3 "most similar" sentences. This is a "brute-force" semantic search engine.

Week 22: Vector Databases

- **Topics:** A critical question: Why does the Week 21 project fail if the list contains 1 *billion* sentences? The "brute-force" similarity search is too slow. This necessitates a specialized database for high-speed vector search. The learner will be introduced to Vector Databases like ChromaDB (a fast, local-first option) and Pinecone (a powerful, cloud-based option).
- **Weekly Project: Scaling Semantic Search.** The learner will re-build the Week 21 project, but this time, they will *not* store the embeddings in a list. They will: 1) Instantiate a ChromaDB collection (or a Pinecone index), 2) Embed and store the 50 (or 500) sentences in the vector database, 3) Instead of a for loop, they will now use the database's built-in query() function. They will see that it returns the same (or similar) results, but it is infinitely more scalable.

Week 23: Retrieval-Augmented Generation (RAG - Part 1: Manual)

- **Topics:** The complete RAG workflow: **Retrieve, Augment, Generate**. RAG is the most common and effective pattern for "grounding" an LLM (i.e., giving it external knowledge) and preventing it from "hallucinating" (making things up).
- **Connecting the Dots:** RAG is not magic. It is a simple "recipe" that combines all the previous weeks:
    - **Retrieve:** Use a Vector Database (Week 22) to find relevant information.
    - **Augment:** Use Python string formatting (Week 1) to build a new, "augmented" prompt.
    - **Generate:** Send this prompt to an LLM via an API call (Week 6), like the OpenAI API.
- **Weekly Project: "Chat with a Document" (Manual).** The learner will: 1) Take a single text document (e.g., a short news article), 2) "Chunk" it into small, overlapping sentences or paragraphs , 3) Embed and store these chunks in their Vector DB , 4) Write a script that takes a user question, 5) Queries the DB to find the most *relevant* chunks, 6) *Manually* builds a prompt (e.g., f"Context: {retrieved_chunks}\n\nQuestion:

{user_question}\n\nAnswer:"), and 7) Sends this "augmented" prompt to an LLM API (like OpenAI's) and prints the response.

Week 24: RAG (Part 2: Frameworks)

- **Topics:** A reflection: Why was the Week 23 project so much manual work? (Chunking, embedding, querying, prompt-building...). This introduces the need for AI "orchestration frameworks" like LangChain or LlamaIndex, which automate this entire RAG pipeline.
- **Weekly Project: "Chat with a Document" (Automated).** The learner will re-build the *entire* Week 23 project. However, this time, they will do it in ~10-15 lines of code using LangChain. They will use a LangChain "loader" to read the document, a "text splitter" to chunk it, a "vector store" object (that wraps their DB), and a "retrieval chain" to do all the work. This project teaches the value of frameworks and how "real-world" AI applications are actually built. **This RAG-bot will be the second key asset for the deployment phase.**

Part 5: Phase 5 (Weeks 25-30): MLOps, Deployment & Capstone Project

A model that only runs in a Jupyter Notebook is a "hobby." A model that is accessible via a scalable API is a "product." This final phase is a single, 6-week capstone project. It covers the *critical* and *highly-sought-after* skills of MLOps (Machine Learning Operations): turning a model into a robust, scalable, and automated service that real users can access. This phase is the-end-to-end culmination of all previous learning.
Capstone Project Goal (Weeks 25-30): "End-to-End Deployed AI Service" The learner will choose *one* of their two primary assets to take to full production:

1. **Project A:** The Week 18 MNIST Classifier (a classical Deep Learning model).
2. **Project B:** The Week 24 RAG-bot (a modern Generative AI system).

Week 25 (The API):

- **Task:** Build a **FastAPI** web server. FastAPI is a modern, high-performance Python framework for building APIs. The learner will create a /predict endpoint that:
    - (For MNIST): Accepts an image file, loads the saved PyTorch model, performs inference, and returns a JSON response: {"digit": 7, "confidence": 0.98}.
    - (For RAG-bot): Accepts a JSON request {"question": "..."}, passes it to the LangChain RAG-bot, and returns a JSON response: {"answer": "..."}. This step effectively separates the "model logic" from the "application logic."

Week 26 (The Container):

- **Task:** Package the FastAPI application into a **Docker** container. The learner will write a Dockerfile that defines all the instructions to build a self-contained "box." This "box" (image) will bundle:
    1. A base Linux operating system.
    2. The Python interpreter.
    3. All Python libraries from requirements.txt.

4. The application code (the FastAPI server).
5. The model files (e.g., the .pt model file for MNIST or the ChromaDB data for the RAG-bot). The result is a portable, self-contained application that will run *identically* on any machine.

Week 27 (The Cloud):

- **Task:** Deploy the Docker container from Week 26 to a major cloud platform, making the AI service publicly accessible. The learner will choose *one* of the "Big 3" AI platforms to learn:
    - **Option 1 (AWS):** Deploy the container to **Amazon SageMaker**. This involves pushing the Docker image to Amazon ECR (Elastic Container Registry) and creating a SageMaker endpoint to serve it.
    - **Option 2 (GCP):** Deploy to **Google Cloud Vertex AI**. This involves pushing the image to the Artifact Registry and deploying it as a model on a Vertex AI Endpoint.
    - **Option 3 (Azure):** Deploy to **Azure Machine Learning**. This involves registering the model and deploying it as a "managed online endpoint".
    - **Result:** The AI service will now have a public URL. The learner can use a tool like curl or a Python script from *anywhere in the world* to call their API and get a prediction.

Week 28 (The Automation - CI/CD):

- **Task:** Automate the entire deployment process. The learner will create a **GitHub Actions** workflow. This is the core of Continuous Integration/Continuous Deployment (CI/CD) for MLOps. They will write a .yml file in their GitHub repository that triggers *automatically* every time they git push new code. This workflow will:
    1. **CI (Integration):** Run automated tests (e.g., check that the API starts).
    2. **CD (Deployment):** If tests pass, it will automatically build the new Docker image, push it to the cloud (e.g., AWS ECR), and deploy the new version to the cloud endpoint (e.g., SageMaker). This demonstrates an advanced, production-grade MLOps setup.

Week 29 (The Portfolio):

- **Task:** Create a "production-grade" **GitHub Portfolio**. An AI Engineer's portfolio is their most important asset. The learner will go back to their Capstone Project repository and write a comprehensive README.md file. This is not a "code dump"; it is a "sales page" for their skills. The README.md *must* include:
    1. **Project Title:** e.g., "End-to-End RAG Chatbot for".
    2. **Business Problem:** A clear 1-2 sentence description (e.g., "Answering user questions about a complex technical manual").
    3. **AI Solution:** A simple architecture diagram (which can be made in Mermaid) showing the flow: User -> API -> Docker -> RAG -> LLM -> Response.
    4. **Tech Stack:** A list of icons/badges (e.g., Python, FastAPI, Docker, AWS SageMaker, LangChain).
    5. **Live API Endpoint:** The public URL from Week 27.
    6. **How to Use:** A code snippet (using curl or Python requests) showing *exactly* how

to call the live API.

Week 30 (The Interface):

- **Task:** (Optional but highly recommended) Build a simple web interface to "talk" to the deployed API. This provides a visual, interactive demo that is far more impressive in interviews than a code snippet.
    - **Option 1 (Easy):** Use **Streamlit**. Streamlit allows building beautiful data apps in pure Python. The learner can write a simple script that creates a text box, and when the user hits "Enter," it calls the deployed API (from Week 27) and displays the answer.
    - **Option 2 (Standard):** Build a basic **HTML/JavaScript** page. This page will have a text input and a button. The JavaScript's fetch() function will call the deployed API endpoint. This final step completes the "end-to-end" journey, from a blank Python file to a fully deployed, interactive AI application.

Part 7: Life as an AI Engineer: Daily Habits & Continuous Learning

This 7-month roadmap is designed to secure the *first* job. The following habits and strategies are essential for building a successful, long-term career in a field that evolves at an unprecedented pace.

Your New Day-to-Day: What Does an AI Engineer Actually Do?

A common misconception is that an AI Engineer's job is 100% "heads-down coding" of new models. The reality of a day-to-day role, especially in a team environment, is more varied and systems-oriented.

- **Morning :** The day often begins not with new code, but with analysis. This includes "analyzing training logs, comparing model performances" from overnight runs, "preparing summaries" for the team, and participating in "code reviews" (both giving and receiving feedback on colleagues' code).
- **Mid-day :** The core of the day is a mix of "heads-down coding" and "collaborative problem-solving." However, this is heavily interspersed with *meetings*. In large tech companies, this can be 50% or more of the day, including 1:1s, design reviews, and writing "design docs" and proposals.
- **Constant Task :** A primary job of an AI Engineer is "reviewing and interpreting output and telemetry data." Modern AI models, especially LLMs, are often "mysterious". A key, high-value skill is "picking apart apparently inexplicable results" to find the root cause of a failure, a bias, or a "hallucination."
- **The Real Job:** The job is not just "building models"; it's "systems-thinking". It involves "coaching" the AI with better data and prompts , analyzing its failures , and integrating it as a reliable, predictable component in a much larger software product.

How to Track Your Progress (Beyond "Percent Done")

During this 30-week plan, the learner must track progress like a professional AI project, not a simple "to-do" list. This means moving beyond "hours studied" and tracking "Key Performance Indicators" (KPIs) for each weekly project.

- **Metrics for Learning :** * **Model Accuracy / Performance:** Did the Week 18 MNIST model hit 98%? What was the final precision/recall for the Week 14 Spam Classifier?
    - **Training Time:** Did a change in the code make the model train faster?
    - **Deployment Milestones:** Binary "Done/Not Done" for key MLOps tasks. (e.g., "API built," "Dockerized," "Cloud Deployed," "CI/CD Pipeline Green").
- **Tools for Tracking :** Professionals use "Experiment Tracking" tools like MLflow or Neptune.ai. A beginner can start by creating a simple "Experiment Log" (e.g., a spreadsheet or a README.md in each project folder). For each project, this log should record:
    - **Hypothesis:** The goal (e.g., "A Random Forest will be more accurate than Logistic Regression for spam").
    - **Parameters:** The settings (e.g., "Random Forest with 100 trees, TfidfVectorizer").
    - **Results:** The "metrics" (e.g., "95% accuracy, 88% recall").
    - **Conclusion:** What was learned (e.g., "RF was more accurate but slower to train"). This instills the "scientific method" that is the foundation of all AI and machine learning.

Staying Current: The AI Field Changes Every 30 Days

This 7-month roadmap provides the *foundational skills* to get a first job. The *meta-skill* required to keep that job is the ability to learn and adapt continuously. The field is changing weekly, not yearly.

- **Daily:** Filter the noise. Follow a curated list of key researchers, engineers, and news sources on platforms like X (Twitter) and Reddit (e.g., r/learnmachinelearning, r/mlops, r/MachineLearning).
- **Weekly:** Read high-level summaries of new, important research. Sources like "ML-Papers-of-the-Week" digests are invaluable for spotting new trends (e.g., new "agentic" frameworks, new types of models, etc.).
- **Monthly:** "Practice the practice." Pick *one* new tool, paper, or technique that appeared in the weekly digests and spend a weekend replicating it with a mini-project. This "practice of learning" is what separates a good AI Engineer from a great one.

Appendix: The 30-Week AI Engineer Execution Plan

This table synthesizes the entire 7-month curriculum into a single, actionable checklist, providing the granular "day-to-day" and "weekly project" plan requested.

Week	Phase & Key Topic	Daily Tasks (Mon-Thurs)	Weekly Project (Fri-Sun)	Key Concepts to Master
W1	**Phase 1: Python Basics**	**Mon:** Setup (Python, VSCode). Run print("Hello World"). **Tue:** Learn variables (int, str, float), print(), f-strings. **Wed:** Learn functions (def),	**Simple CLI Calculator:** Prompt user for 2 numbers and an operator. Return the result.	Variables, Data Types, Functions, Conditionals

Week	Phase & Key Topic	Daily Tasks (Mon-Thurs)	Weekly Project (Fri-Sun)	Key Concepts to Master
		parameters, return values. **Thu:** Learn if/elif/else, comparison operators.		
W2	**Phase 1: Python Data Structures**	**Mon:** Master Lists (methods: .append(), .pop(), slicing). **Tue:** Master Dictionaries (keys, values, .get()). **Wed:** Master Sets (uniqueness) and Tuples (immutability). **Thu:** Learn for loops, while loops, range().	**Text-Based Hangman Game:** Use a List for words, a Set for guesses, and a while loop for the game.	Lists, Dictionaries, Sets, Tuples, Loops
W3	**Phase 1: Object-Oriented Programming (OOP)**	**Mon:** Read about OOP. Understand class, object, attribute, method. **Tue:** Learn the __init__() constructor (self). **Wed:** Build your first class (e.g., Dog with .bark() method). **Thu:** Learn inheritance (Parent/Child classes).	**Simple Banking App:** Create Customer class (name, PIN) and Account class (customer, balance, deposit(), withdraw()).	Class vs. Object, __init__, self, Attribute, Method
W4	**Phase 1: Algorithms & Linear Algebra Intuition**	**Mon:** Implement Bubble Sort to understand O(n^2). **Tue:** Learn what a Vector, Matrix, and Tensor are. **Wed:** Read: How data (features, datasets) is *represented* by matrices. **Thu:**	**Visualizing Linear Algebra:** Plot a vector. Define a 2x2 matrix. Multiply vector by matrix. Plot the *new* vector.	Algorithm Efficiency, Vector, Matrix, Tensor, Linear Transformation

Week	Phase & Key Topic	Daily Tasks (Mon-Thurs)	Weekly Project (Fri-Sun)	Key Concepts to Master
		Install Matplotlib. Learn to plot a 2D vector.		
W5	**Phase 1: Calculus & Statistics Intuition**	**Mon:** Read: What is a derivative (slope)? What is a gradient? **Tue:** Read: What is the Chain Rule (at a high level)? **Wed:** Learn Stats: Mean, Median, Mode (centrality). **Thu:** Learn Stats: Variance, Standard Deviation (spread).	**Gradient Descent from Scratch:** Code y = x^2. Use its derivative (2x) to "walk" from a random x down to the minimum at x=0.	Gradient, Derivative, Gradient Descent, Mean, Variance
W6	**Phase 1: Files & Public APIs**	**Mon:** Learn to open(), read(), and write() .txt files. **Tue:** Learn to read/write .csv files with the csv module. **Wed:** Install requests. Learn GET requests. **Thu:** Learn what JSON is and how to parse it (it's just a Python dictionary).	**Public API "Weather App":** Prompt user for a city. Call a free weather API. Parse the JSON response. Print the forecast.	File I/O, API, requests, JSON
W7	**Phase 2: NumPy**	**Mon:** Install NumPy. Understand the ndarray. **Tue:** Master ndarray indexing, slicing, and filtering. **Wed:** Understand Vectorization (NumPy vs. for loops). **Thu:** Learn broadcasting and universal functions (np.mean(),	**Image Manipulation with NumPy:** Load an image. Use array operations to change brightness, contrast, and crop it.	ndarray, Vectorization, Slicing, Broadcasting

Week	Phase & Key Topic	Daily Tasks (Mon-Thurs)	Weekly Project (Fri-Sun)	Key Concepts to Master
		np.sum()).		
W8	**Phase 2: Pandas (Cleaning)**	**Mon:** Install Pandas. Learn Series and DataFrame. **Tue:** Master pd.read_csv(). Inspect data with .head(), .info(), .describe(). **Wed:** Find missing data (.isnull()). Handle it (.dropna(), .fillna()). **Thu:** Clean data (.astype(), .str.replace(), .drop_duplicates()).	**Real-World Data Cleaning:** Take a messy dataset. Clean missing values, types, and duplicates. Save the clean.csv.	DataFrame, Series, read_csv, fillna, dropna
W9	**Phase 2: Pandas (Analysis)**	**Mon:** Master filtering (loc, iloc, boolean indexing). **Tue:** Master grouping (.groupby()). **Wed:** Master aggregations (.mean(), .sum(), .count(), .value_counts()). **Thu:** Learn to merge() and join() two DataFrames.	**EDA on Telecom Churn:** Load churn data. Use .groupby() and aggregations to answer 3-4 business questions.	loc/iloc, groupby, Aggregation, merge
W10	**Phase 2: Matplotlib & Seaborn**	**Mon:** Install Matplotlib. Make a simple .plot() and .bar() chart. **Tue:** Customize plots (titles, labels, legends). **Wed:** Install Seaborn. Make a .boxplot() and .heatmap(). **Thu:** Understand *when* to use each	**EDA Visualization Dashboard:** In a Jupyter Notebook, create 4-5 plots (hist, bar, heatmap) to visualize your findings from Week 9.	plt.plot(), plt.hist(), sns.heatmap(), sns.boxplot()

Week	Phase & Key Topic	Daily Tasks (Mon-Thurs)	Weekly Project (Fri-Sun)	Key Concepts to Master
		plot (hist, bar, scatter, box).		
W11	**Phase 2: Scikit-Learn (First Model)**	**Mon:** Install scikit-learn. Learn the .fit() / .predict() API. **Tue:** Understand Supervised Learning. Learn train_test_split. **Wed:** Learn Linear Regression (for numbers). **Thu:** Learn Logistic Regression (for categories).	**House Price Prediction (Regression):** Use Linear Regression to predict house prices. Evaluate with Mean Squared Error (MSE).	fit, predict, train_test_split, Linear Regression, MSE
W12	**Phase 2: Scikit-Learn (Evaluation)**	**Mon:** Learn *why* "Accuracy" can be misleading. **Tue:** Master the Confusion Matrix. **Wed:** Define and calculate: Accuracy, Precision, Recall, F1-score. **Thu:** Learn what Cross-Validation is and how to use cross_val_score.	**Breast Cancer Classifier (Logistic Regression):** Build a classifier. Report Accuracy, Precision, and Recall. Re-run with cross-validation.	Accuracy, Precision, Recall, Confusion Matrix, Cross-Validation
W13	**Phase 3: Supervised Learning (Trees)**	**Mon:** Read about Decision Trees (how they learn if/else rules). **Tue:** Learn TfidfVectorizer (converts text to numbers). **Wed:** Read about Random Forests (Ensemble "voting"). **Thu:** Train a RandomForestClassifier.	**Spam Classifier (Part 1):** Use TfidfVectorizer and RandomForestClassifier to build a spam detector. Evaluate it.	Decision Tree, Random Forest, TfidfVectorizer, Ensemble

Week	Phase & Key Topic	Daily Tasks (Mon-Thurs)	Weekly Project (Fri-Sun)	Key Concepts to Master
W14	**Phase 3: Supervised Learning (Other)**	**Mon:** Learn Support Vector Machines (SVMs) (geometric intuition). **Tue:** Learn K-Nearest Neighbors (KNN) (voting intuition). **Wed:** Learn Naive Bayes (probabilistic intuition). **Thu:** Practice selecting models for different problems.	**Spam Classifier (Part 2):** Re-build the spam classifier with MultinomialNB and LinearSVC. Create a table comparing all 3 models.	SVM, KNN, Naive Bayes, Model Selection
W15	**Phase 3: Unsupervised Learning (Clustering)**	**Mon:** Understand Unsupervised vs. Supervised learning. **Tue:** Learn the K-Means Clustering algorithm (centroids, iteration). **Wed:** Learn the "Elbow Method" to find the best k. **Thu:** Train a KMeans model in Scikit-learn.	**Customer Segmentation:** Use K-Means on the "Mall Customer" dataset. Use the Elbow Method to pick k. Analyze the resulting clusters.	Unsupervised Learning, K-Means, Clustering, Elbow Method
W16	**Phase 3: Unsupervised Learning (Dim. Reduction)**	**Mon:** Read about the "Curse of Dimensionality." **Tue:** Learn what PCA (Principal Component Analysis) does. **Wed:** Connect PCA to Week 4 (Eigenvectors = "principal components"). **Thu:** Train a PCA model in Scikit-learn. Set	**Visualizing Customer Segments:** Take the Week 15 data. Use PCA to reduce it to 2D. Make a scatter plot, coloring by cluster ID.	PCA, Dimensionality Reduction, Eigenvectors

Week	Phase & Key Topic	Daily Tasks (Mon-Thurs)	Weekly Project (Fri-Sun)	Key Concepts to Master
		n_components=2.		
W17	**Phase 3: Deep Learning (Intro & ANNs)**	**Mon:** Install PyTorch. Read PyTorch vs. TensorFlow. **Tue:** Learn the nn.Module, nn.Linear, Activation Functions (ReLU). **Wed:** Learn the PyTorch training loop (loss, optimizer, backprop). **Thu:** Build a simple ANN for the Iris dataset.	**Iris Classifier with an ANN:** Build and train your first neural network from scratch in PyTorch to classify the Iris dataset.	PyTorch, nn.Module, ANN/MLP, Activation Function, Training Loop
W18	**Phase 3: Deep Learning (CNNs)**	**Mon:** Read *why* ANNs fail on images. **Tue:** Learn the nn.Conv2d (Convolution) layer (filters). **Wed:** Learn the nn.MaxPool2d (Pooling) layer (downsampling). **Thu:** Build a simple CNN architecture (Conv -> Pool -> Conv -> Pool -> FC).	**Handwritten Digit Recognition (MNIST):** Build a CNN in PyTorch to classify MNIST digits. Aim for >98% accuracy. **Save this model!**	CNN, Convolution Layer, Pooling Layer, MNIST
W19	**Phase 4: Hugging Face (The Easy Way)**	**Mon:** Create a Hugging Face (HF) account. **Tue:** Read about the Transformer architecture (high-level). **Wed:** Install transformers. Learn the pipeline() function.	**One-Line AI Tools:** Build a script that uses the HF pipeline() to perform 3 tasks: Sentiment Analysis, Text Summarization, and Image Captioning.	Hugging Face, Transformers, pipeline()

Week	Phase & Key Topic	Daily Tasks (Mon-Thurs)	Weekly Project (Fri-Sun)	Key Concepts to Master
		Thu: Use pipeline() for sentiment analysis and text summarization.		
W20	**Phase 4: Hugging Face (The "Real" Way)**	**Mon:** Learn what a Tokenizer is. **Tue:** Load a pre-trained AutoTokenizer. **Wed:** Load a pre-trained AutoModel. **Thu:** Understand how to feed tokens (input_ids, attention_mask) to a model and interpret the output "logits".	**Re-Build the Sentiment Analyzer:** Replicate the Week 19 project *without* pipeline(). Manually load the tokenizer and model.	AutoTokenizer, AutoModel, Tokens, Logits, input_ids
W21	**Phase 4: Embeddings & Semantic Search**	**Mon:** Learn what an "Embedding" is (a vector of meaning). **Tue:** Install sentence-transformers. Load a model. **Wed:** Learn Cosine Similarity (the math for "closeness"). **Thu:** Write a Python function for cosine similarity.	**Semantic Search Engine (Local):** Embed 50 sentences. Write a function that takes a query, embeds it, and uses a for loop to find the most similar sentence.	Embeddings, Semantic Search, Cosine Similarity
W22	**Phase 4: Vector Databases**	**Mon:** Read: Why does Week 21 fail at 1 billion sentences? **Tue:** Install chromadb. Learn the basic API. **Wed:** Create a "collection." Add embeddings and text. **Thu:** Use the collection.query()	**Scaling Semantic Search:** Re-build the Week 21 project. Store the embeddings in ChromaDB. Use collection.query() instead of a for loop.	Vector Database, ChromaDB, Pinecone, Scalable Search

Week	Phase & Key Topic	Daily Tasks (Mon-Thurs)	Weekly Project (Fri-Sun)	Key Concepts to Master
		function to find similar vectors.		
W23	**Phase 4: RAG (Part 1: Manual)**	**Mon:** Learn the RAG workflow: Retrieve, Augment, Generate. **Tue:** Learn "Chunking" (splitting big docs). **Wed:** Get an OpenAI (or other LLM) API key. **Thu:** Practice manually creating a prompt with f-strings.	**"Chat with a Document" (Manual):** Chunk a doc, store in ChromaDB. Take a query, query() ChromaDB (Retrieve), build a prompt (Augment), send to OpenAI (Generate).	RAG, Retrieve, Augment, Generate, Chunking, Prompt Engineering
W24	**Phase 4: RAG (Part 2: Frameworks)**	**Mon:** Read: Why was Week 23 so much work? **Tue:** Install langchain. Read the high-level concepts. **Wed:** Learn LangChain's "Loaders," "Splitters," and "Vector Stores." **Thu:** Learn how to build a RetrievalQA chain.	**"Chat with a Document" (Automated):** Re-build the *entire* Week 23 project using a LangChain chain. Do it in ~10 lines of code.	LangChain, LlamaIndex, Orchestration, AI Pipelines
W25	**Phase 5: Capstone (The API)**	**Mon:** Install fastapi and uvicorn. **Tue:** Create a simple "hello world" API. **Wed:** Design the /predict endpoint (request/response models). **Thu:** Write the API logic to load your model (MNIST or RAG) and return a JSON prediction.	**Build FastAPI Server:** Create a main.py with a FastAPI app. Load your saved model. Create a /predict endpoint.	FastAPI, Uvicorn, API, Endpoint, JSON
W26	**Phase 5:**	**Mon:** Install	**Dockerize the AI**	Docker, Dockerfile,

Week	Phase & Key Topic	Daily Tasks (Mon-Thurs)	Weekly Project (Fri-Sun)	Key Concepts to Master
	Capstone (The Container)	Docker. Run docker run hello-world. **Tue:** Learn Dockerfile syntax (FROM, COPY, RUN, CMD). **Wed:** Write a Dockerfile for your FastAPI app. **Thu:** Build the image (docker build) and run it (docker run). Test it.	**App:** Write a Dockerfile that packages your FastAPI app, requirements.txt, and model file into a runnable container.	Container, Image, Port Mapping
W27	**Phase 5: Capstone (The Cloud)**	**Mon:** Create an account (AWS, GCP, or Azure). **Tue:** Learn the chosen platform's container registry (ECR, Artifact Registry, ACR). **Wed:** Learn the model serving tool (SageMaker, Vertex AI, Azure ML). **Thu:** Push your Docker image to the registry.	**Deploy to the Cloud:** Deploy your Docker image to a managed cloud endpoint (e.g., SageMaker or Vertex AI). Get a public API URL.	AWS SageMaker, GCP Vertex AI, Azure ML, Cloud Deployment
W28	**Phase 5: Capstone (The Automation)**	**Mon:** Read about CI/CD. **Tue:** Learn GitHub Actions basics (workflows, jobs, steps). **Wed:** Find a "Deploy to SageMaker/Vertex " GitHub Action. **Thu:** Create the .yml file in .github/workflows/.	**Create CI/CD Pipeline:** Build a GitHub Action that *automatically* (on git push) builds your Docker image and deploys it to the cloud endpoint.	CI/CD, GitHub Actions, MLOps, Automation
W29	**Phase 5: Capstone (The Portfolio)**	**Mon:** Read about good GitHub README.md files. **Tue:** Create an architecture	**Build GitHub Portfolio:** Write a "sales page" README.md for your capstone	Portfolio, README.md, Documentation

Week	Phase & Key Topic	Daily Tasks (Mon-Thurs)	Weekly Project (Fri-Sun)	Key Concepts to Master
		diagram for your project. **Wed:** Write the "Business Problem" and "AI Solution" sections. **Thu:** Add the "Tech Stack" and "How to Use" (with code!) sections.	project. Include the live API URL and code examples.	
W30	**Phase 5: Capstone (The Interface)**	**Mon:** Install streamlit. **Tue:** Create a "hello world" Streamlit app. **Wed:** Add a text input box and a button. **Thu:** Write the requests code to call your *deployed cloud API* from the Streamlit app.	**Build a Streamlit UI:** Create a simple web app that provides a UI for your deployed model. Host it on Streamlit Community Cloud.	Streamlit, Frontend, UI, Full-Stack AI

Final Report (TXT)

A TXT file version of this report can be generated by copying the text and flowchart.

The 30-Week Plan: A Zero-to-Production AI Engineer Roadmap

Part 1: The Modern AI Engineer: A 7-Month Battle Plan

Introduction: The 7-Month (30-Week) Challenge

This report details an intensive, 30-week curriculum designed to transition a dedicated individual from an absolute beginner to a job-ready Junior AI Engineer. This timeline is aggressive and assumes a "medium to fast learner" (as per the user query) who can commit to an "intense daily study routine" of 6 to 8 hours per day.

This 7-month (approximately 30-week) plan is predicated on consistent, disciplined effort. It is a "grind" , but one that is demonstrably achievable. While some industry experts describe a "3-6 year journey" to become a *senior* or *lead* AI engineer at a top company , other documented cases show individuals going from zero programming experience to a "Junior Gen AI Engineer" role in as little as 5.5 months.

This roadmap reconciles these timelines by focusing on a specific, modern, and in-demand role.

The 7-month plan is not a path to senior-level, theoretical research; it is a pragmatic, project-based curriculum aimed squarely at a high-value *entry-level* position in the 2025 job market. This plan aligns with 6-to-12-month roadmaps that structure learning from foundational programming to advanced specialization and deployment.

The 2025 Job Title: AI Engineer vs. ML Engineer

Understanding the entire strategy of this 30-week plan requires a critical distinction between two roles that are often confused: the Machine Learning (ML) Engineer and the Artificial Intelligence (AI) Engineer.

- **The Machine Learning (ML) Engineer (Traditional):** This role is "model-focused". The ML engineer is primarily concerned with *designing, building, and scaling ML models from scratch*. Their daily work involves "data preprocessing, classical ML, predictive models, deployment pipelines," and a deep algorithmic understanding of how to train these models.
- **The AI Engineer (Modern, 2025):** This role is "system-focused". The modern AI Engineer, particularly in the post-LLM (Large Language Model) era, focuses on *using* and *integrating* existing, powerful, pre-trained models to build applications. Their world revolves around "LLMs, prompt engineering, fine-tuning, generative models," and Retrieval-Augmented Generation (RAG).

As some practitioners have noted, the job landscape has fundamentally shifted; "LLM changed the ML jobs forever". The traditional path of "AI/ML as intended before, where you needed to do a lot of modeling and fine tuning is dying fast" for many roles, being replaced by engineers who can effectively *wield* these new generative tools.
Therefore, this 30-week roadmap is strategically designed for the *AI Engineer* role. It builds a necessary foundation in classical ML (Phases 2-3) to provide context and intuition, but it then *pivots hard and fast* to the modern Generative AI stack (Phase 4). This pivot is the most efficient path to acquiring the high-value, "real-world" skills that employers are hiring for *today*.

The 30-Week Roadmap: An Interactive Flowchart

The following flowchart provides a high-level visual of the five-phase, 30-week journey from beginner to deployed AI Engineer.
graph TD A --> B(Phase 1: Python & Math Foundations Weeks 1-6); B --> C(Phase 2: Data Scientist's Toolkit Weeks 7-12); C --> D(Phase 3: Classical ML & DL Foundations Weeks 13-18); D --> E(Phase 4: The Modern GenAI Stack Weeks 19-24); E --> F(Phase 5: MLOps & Deployment Weeks 25-30); F --> G[Hired: Junior AI Engineer];
subgraph Phase 1 B1 B2 B3 end
subgraph Phase 2 C1 C2 C3 C4 end
subgraph Phase 3 D1 D2 D3 end
subgraph Phase 4 E1 E2 E3 end
subgraph Phase 5 F1 F2 F3 F4 F5 end
B1-->B2-->B3; C1-->C2-->C3-->C4; D1-->D2-->D3; E1-->E2-->E3; F1-->F2-->F3-->F4-->F5;

Part 2: Phase 1 (Weeks 1-6): The Python & Foundational Math Vanguard

This initial phase is about forging the fundamental tools. The primary weapon of the AI Engineer is Python, and this phase is dedicated to mastering its syntax, logic, and structure. This includes the often-challenging but non-negotiable topic of Object-Oriented Programming (OOP). Concurrently, this phase builds the essential *intuition* for the core mathematics that power AI, approaching it not as an academic abstract but as the practical language of data.

Week 1: Python Basics

- **Topics:** Foundational Python syntax. This includes understanding and using variables (strings, integers, floats), the print() function for output, creating reusable blocks of code with functions, implementing logic with if/else conditions, and iterating with for and while loops.
- **Weekly Project: Simple CLI Calculator.** The learner will build a Python script that runs in the command line. The script will prompt the user for two numbers and an operator (e.g., +, -, *, /). It will then use a function and conditional logic to perform the calculation and print the result. This project solidifies mastery of variables, user input, functions, and if/else statements.

Week 2: Python Data Structures

- **Topics:** Storing collections of related data using Python's four built-in data structures: Lists, Dictionaries, Sets, and Tuples. The focus is on *when* to use each: Lists for ordered, mutable collections; Dictionaries for key-value (associative) lookups; Sets for unique, unordered items; and Tuples for immutable, fixed collections.
- **Weekly Project: Text-Based Hangman Game.** The learner will build a complete Hangman game. This project is a perfect test of data structures. It requires a List to hold the possible secret words, a Set to store the letters the user has already guessed (preventing duplicate guesses), and a List to represent the "blanks" of the word being guessed. The game loop (while) will check the game state, demonstrating practical application of all Week 1 and 2 concepts.

Week 3: Object-Oriented Programming (OOP)

- **Topics:** The principles of Object-Oriented Programming (OOP). This includes defining "blueprints" with Classes, creating "instances" called objects, understanding the __init__() constructor, bundling data as attributes, and defining behaviors with methods. The concept of inheritance, where a new class can "inherit" attributes and methods from a parent class, will also be introduced.
- **Why This is Critical:** Many beginners struggle with OOP. However, it is non-negotiable. All major AI libraries, including Pandas, NumPy, and TensorFlow, are built using the OOP paradigm. To use these tools effectively (and not just copy-paste code), one must understand how objects store data and expose methods.
- **Weekly Project: Simple Banking App.** The learner will create two classes. First, a Customer class with attributes like name and PIN. Second, an Account class that holds a Customer object as an attribute, along with a balance. The Account class will have methods like deposit(), withdraw() (which checks the balance), and check_balance(). This project replicates real-world software design, where data (attributes) and behaviors (methods) are bundled together logically.

Week 4: Algorithms & Linear Algebra Intuition

- **Topics:**
    - **Algorithms:** Implementing basic sorting algorithms like Bubble Sort or Insertion Sort. The goal is not to master sorting, but to understand algorithm *efficiency* (Big O notation) and the process of turning a logical procedure into code.
    - **Linear Algebra:** This is the *language of data*. The focus is on visual intuition, not abstract proofs. The learner will understand Vectors (a single feature list), Matrices (a whole dataset or an image), and Tensors (the general data structure).
- **Weekly Project: Visualizing Linear Algebra.** Using a plotting library (Matplotlib will be introduced early for this), the learner will plot vectors as arrows on a 2D graph. They will then define a 2x2 matrix and write a function to multiply their vectors by this matrix. By plotting the *original* and *transformed* vectors, they will visually see how matrix multiplication *transforms* (scales, rotates, shears) space. This builds the fundamental visual intuition for what PCA and SVD (Eigenvectors) are actually doing.

Week 5: Calculus & Statistics Intuition

- **Topics:**
    - **Calculus:** This is the *engine* of optimization. The key concepts are Gradients (the "slope" of an error hill), Derivatives (the "rate of change" that gives us the slope), and the Chain Rule (the mechanism for backpropagation).
    - **Statistics:** This is the *backbone* of machine learning. The focus is on descriptive statistics: Mean, Median, Mode (central tendency), Variance, Standard Deviation (data spread), and basic Probability Distributions (Normal, Binomial).
- **Weekly Project: Implementing Gradient Descent from Scratch.** The learner will define a simple mathematical function in Python, such as y = x^2. They will also write a function for its derivative, dy/dx = 2x. The script will start with a random value for x. It will then repeatedly: 1) calculate the gradient (derivative) at that x, 2) take a small step in the *opposite* direction of the gradient. The script will print the value of x and y at each step, showing how it "walks" down the curve to find the minimum y. This project is the single most important concept for understanding *all* model training.

Week 6: Working with Files & Public APIs

- **Topics:** Bridging the gap from local scripts to the outside world. This includes reading and writing data from/to text (.txt) and comma-separated-value (.csv) files. It also introduces making GET requests to public Application Program Interfaces (APIs) using the requests library and parsing the resulting JSON data.
- **Why This is Critical:** This is the bridge to the modern AI Engineer role. AI engineers constantly pull data from various sources and interact with models via API calls.
- **Weekly Project: Public API "Weather App".** The learner will build a command-line tool that prompts the user for a city name. The script will then: 1) Make an API call to a free weather service (like OpenWeatherMap), 2) Receive a JSON response, 3) Parse the JSON to find the relevant keys (like temp and description), and 4) Print a human-readable forecast (e.g., "The weather in London is: light rain with a temperature of 15 C.").

Part 3: Phase 2 (Weeks 7-12): The Data Scientist's Toolkit

With a strong Python foundation, the learner now masters the "Data Stack." This is the day-to-day toolkit for *all* data work, built upon the libraries that form the core of the scientific Python ecosystem. This phase involves learning to manipulate massive datasets efficiently (NumPy, Pandas) , visualize patterns (Matplotlib) , and build the first predictive models (Scikit-learn).

Week 7: NumPy

- **Topics:** The core data structure of all data science: the NumPy ndarray. The learner will master vectorization (why for loops are slow), array indexing and slicing, broadcasting (how NumPy handles operations between arrays of different shapes), and high-speed mathematical operations on matrices.
- **Weekly Project: Image Manipulation with NumPy.** An image is just a 3D NumPy array (width, height, color channels). The learner will load a simple image into an ndarray. They will then use vectorized operations to: 1) Change its brightness (by adding a constant to the entire array), 2) Increase its contrast (by multiplying the array), and 3) Crop the image (using array slicing). This project directly connects the abstract concept of "matrices" to a tangible, visual output.

Week 8: Pandas (Part 1 - Cleaning)

- **Topics:** The DataFrame and Series—the most common objects for working with tabular data. This week focuses on data *cleaning*. Topics include: reading data from CSV/Excel files (pd.read_csv), finding and handling missing data (.dropna(), .fillna([span_170](start_span)[span_170](end_span))), correcting data types (.astype()), and cleaning text fields using string manipulation (.str()).
- **Weekly Project: Real-World Data Cleaning.** The learner will be given a notoriously "messy" real-world dataset (e.g., a movie budget dataset , a city bike dataset , or a Star Wars survey). Their task is to perform a data-cleaning audit. They will load the data, identify all missing values, duplicates, and invalid entries (e.g., a "Yes/No" column with "Yes", "No", and "Y" entries). The final deliverable is a *new, clean* CSV file and a Jupyter Notebook documenting all the changes made.

Week 9: Pandas (Part 2 - Analysis)

- **Topics:** Exploratory Data Analysis (EDA). With clean data, the learner now focuses on *asking questions*. Topics include: filtering data with loc and iloc, grouping data with .groupby(), performing aggregations (.mean(), .sum(), .value_counts()), and merging/joining two separate DataFrames together.
- **Weekly Project: Exploratory Data Analysis (EDA) on Telecom Churn.** Using a classic "Telecom Churn" dataset , the learner will act as a data analyst. They will use Pandas to answer specific business questions: "What is the overall churn rate?" , "Which 'Total day charge' value is most common?", "Do customers with more 'vmail messages' churn

less?", and "What are the average call charges for customers who churned vs. those who didn't?".

Week 10: Matplotlib & Seaborn

- **Topics:** The "grammar" of data visualization. The learner will master the standard plotting libraries. Matplotlib is used for basic, customizable plots (line plots, bar charts, histograms, scatter plots). Seaborn, which is built on Matplotlib, is used for more complex statistical visualizations like correlation heatmaps and boxplots.
- **Weekly Project: EDA Visualization Dashboard.** Building directly on the Week 9 project, the learner will use a Jupyter Notebook to create a visual report. They will take their answers from the previous week and visualize them. This includes: a histogram of "Total day charge," a bar chart of "Churn," a boxplot showing the distribution of "Total day charge" for churned vs. non-churned customers, and a correlation heatmap of all numerical features to see what is most related.

Week 11: Scikit-Learn (Part 1 - First Model)

- **Topics:** Introduction to the "Estimator" API, the simple and consistent interface used by Scikit-learn: .fit(), .predict(). This is the first week of *Supervised Learning*. The learner will split data into training and testing sets (train_test_split) and build and understand the two most fundamental models: Linear Regression (for predicting continuous values) and Logistic Regression (for predicting categories).
- **Weekly Project: House Price Prediction (Regression).** Using a "Boston Housing" or "California Housing" dataset , the learner will build their first end-to-end ML model. They will: 1) Load the data, 2) Split it into training and testing sets, 3) fit() a LinearRegression model on the training data, and 4) predict() prices on the test data. Finally, they will evaluate the model's performance by calculating the Mean Squared Error.

Week 12: Scikit-Learn (Part 2 - Evaluation)

- **Topics:** How to know if a model is *actually* good. This week focuses on classification model evaluation metrics: Accuracy, Precision, Recall, and F1-score. The learner will master the Confusion Matrix, which is the "scoreboard" for a classifier. This week also introduces the concepts of Overfitting (doing well on training data, poorly on test data) and Cross-Validation (a more robust method for evaluating performance).
- **Weekly Project: Breast Cancer Classifier (Logistic Regression).** Using the "Breast Cancer" dataset , the learner will build a LogisticRegression classifier. The project has a specific, multi-step goal: 1) Split the data , 2) Train the model, 3) Evaluate its performance using a confusion matrix and report the accuracy, precision, and recall. 4) *Critically*, they will then re-evaluate the model using 5-fold cross-validation to get a more robust and reliable performance score. This teaches them to be skeptical of a single "accuracy" number.

Part 4: Phase 3 (Weeks 13-18): Classical ML & Deep Learning Foundations

In this phase, the learner moves beyond simple linear models to the "workhorse" algorithms of classical ML. The objective is not to become a world-expert in any single algorithm, but to build a strong *intuition* for *how* different models "think"—some use geometry (SVMs), some use probability (Naive Bayes), and some use logic (Decision Trees). This phase then culminates in the critical leap to Deep Learning, building the foundational neural network architectures (ANNs and CNNs) that power the entire modern AI field.

Week 13: Supervised Learning (Trees)

- **Topics:** Decision Trees (DTs) and their powerful "ensemble" version, Random Forests (RFs). DTs work by learning a series of simple if/else rules from the data. Random Forests improve upon this by building *many* "weak" trees on random subsets of data and having them "vote," which dramatically reduces overfitting.
- **Weekly Project: Spam Classifier (Part 1).** Using a "SMS Spam" dataset , the learner will build a text classifier. This requires a new step: preprocessing the text. The learner will use Scikit-learn's CountVectorizer or TfidfVectorize[span_247](start_span)[span_247](end_span)[span_249](start_span)[span_249](end_span)[span_251](start_span)[span_251](end_span)[span_253](start_span)[span_253](end_span)[span_255](start_span)[span_255](end_span)r to convert text messages into a numerical matrix (a bag-of-words). They will then train a RandomForestClassifier on this matrix and compare its performance to the Logistic Regression model from Week 12.

Week 14: Supervised Learning (Other)

- **Topics:** Expanding the toolkit with other powerful classifiers. This includes:
    - **Support Vector Machines (SVMs):** A geometric model that finds the "widest possible street" (hyperplane) to separate data points.
    - **K-Nearest Neighbors (KNN):** A simple, instance-based model that classifies a new point by a "majority vote" of its k closest neighbors.
    - **Naive Bayes:** A probabilistic classifier that uses Bayes' Theorem to calculate the probability of a class given a set of features.
- **Weekly Project: Spam Classifier (Part 2).** This project teaches the real-world task of model selection. The learner will re-build the spam classifier from Week 13, but this time using a MultinomialNB (Naive Bayes) model and a LinearSVC (SVM) model. They will then create a comparison table (in their notebook) showing the accuracy, precision, and recall for all three models (Random Forest, Naive Bayes, SVM). This demonstrates how to empirically select the best model for a given task.

Week 15: Unsupervised Learning (Clustering)

- **Topics:** Shifting from *Supervised* (labeled data) to *Unsupervised* (unlabeled data) learning. The goal of clustering is to find "natural groups" in the data. The learner will master the K-Means Clustering algorithm, the most popular and intuitive clustering method. This includes learning how to use the "Elbow Method" to find the optimal number of clusters (k).
- **Weekly Project: Customer Segmentation.** Using the "Mall Customer" dataset, which contains features like age, annual income, and spending score , the learner will use

K-Means to cluster customers. They will first use the "Elbow Method" to plot the "inertia" (within-cluster sum-of-squares) for k=1 through k=10 to justify their choice of k (e.g., k=5). The output will be a DataFrame with a new "cluster" column, identifying distinct segments like "high income, low spend" or "low income, high spend."

Week 16: Unsupervised Learning (Dimensionality Reduction)

- **Topics:** The "curse of dimensionality"—why models often fail when given too many features. The learner will master Principal Component Analysis (PCA) as a technique for dimensionality reduction. This connects directly back to Phase 1: PCA works by finding the Eigenvectors (the "principal components") of the data's covariance matrix, which represent the "directions of highest variance".
- **Weekly Project: Visualizing Customer Segments.** This project provides a powerful "Aha!" moment. The customer data from Week 15 is 3D+ (age, income, score). This makes it hard to visualize. The learner will take that data, use PCA to reduce its dimensions from 3D to 2D , and *then* create a 2D scatter plot. The x-axis will be "Principal Component 1" and the y-axis will be "Principal Component 2". They will color the dots on this plot based on the cluster ID from Week 15. For the first time, they will be able to *see* the distinct clusters their algorithm found.

Week 17: Deep Learning (Intro to PyTorch & ANNs)

- **Topics:** The major leap into Deep Learning.
    - **Frameworks:** A discussion of TensorFlow vs. PyTorch. This roadmap selects PyTorch, as it is often praised for its "Pythonic" nature, intuitive syntax, and dominance in research, making it easier for beginners to grasp.
    - **Core Concepts:** The PyTorch nn.Module (the "blueprint" for all models). Artificial Neural Networks (ANNs), also called Multi-Layer Perceptrons (MLPs). The learner will understand the basic components: Layers, Neurons, Weights, Biases, and Activation Functions (like ReLU and Sigmoid).
- **Weekly Project: Iris Classifier with an ANN.** The learner will build their first neural network. They will take the classic "Iris" dataset (which they might have used with Scikit-learn), define a simple ANN (e.g., 4 input features, 1 hidden layer of 10 neurons, 3 output classes), and write the PyTorch training loop (forward pass, loss calculation, backpropagation, optimizer step). This replaces a Scikit-learn model with a custom-built neural network.

Week 18: Deep Learning (Convolutional Neural Networks - CNNs)

- **Topics:** Why ANNs (which are "fully connected") fail on image data. Introduction to the core concepts of Convolutional Neural Networks (CNNs), the architecture that powers most computer vision. The learner will master the two key layer types:
    - **Convolution Layers:** These act as "feature detectors" (filters) that scan for edges, shapes, and textures.
    - **Pooling Layers:** These "downsample" the image, reducing its size while keeping the most important information.
- **Weekly Project: Handwritten Digit Recognition (MNIST).** This is the foundational "Hello, World!" project of deep learning. The learner will build a simple CNN in PyTorch to

classify the "MNIST" dataset of handwritten digits (0-9). They will define a model with 2-3 convolution/pooling blocks followed by a fully connected (ANN) "head." They will train this model and watch it achieve >98% accuracy on data it has never seen. **This trained model will be a key asset saved for the deployment phase.**

Part 5: Phase 4 (Weeks 19-24): The Modern Generative AI Stack

This phase is the *pivot*. Having built a solid foundation in classical ML and the *architecture* of deep learning, the learner now "graduates" from building small models from scratch to *using* massive, state-of-the-art, pre-trained models. This is the core of the 2025 AI Engineer role. This phase is entirely dedicated to the "Generative" stack: Hugging Face (the model hub) , Embeddings (the language of meaning) , Vector Databases (the "memory" for AI) , and Retrieval-Augmented Generation (RAG) (the system that ties it all together).

Week 19: Transformers & Hugging Face (The Easy Way)

- **Topics:** A high-level introduction to the Transformer architecture (which powers models like GPT and BERT). An introduction to the Hugging Face (HF) ecosystem, which is the "GitHub for AI models". The focus is on the simple, high-level pipeli[span_337](start_span)[span_337](end_span)ne() function, which abstracts all the complexity.
- **Weekly Project: One-Line AI Tools.** The learner will build a simple Python script that demonstrates the power of the HF pipeline(). In just a few lines of code each, they will implement:
    1. A Sentiment Analysis tool.
    2. A Text Summarization tool.
    3. An Image Captioning tool. This project's goal is to produce a "wow" moment, demonstrating the immense power available "off-the-shelf".

Week 20: Hugging Face (The "Real" Way)

- **Topics:** Going "under the hood" of the pipeline(). The learner will be introduced to the two key components:
    1. AutoTokenizer: The object that converts human-readable text into numerical "tokens" that a model understands.
    2. AutoModel: The actual pre-trained model (e.g., BERT) loaded from the hub. The learner will understand what "tokens" and "attention masks" are and why they are necessary.
- **Weekly Project: Re-Build the Sentiment Analyzer.** The learner will replicate the Week 19 project, but *without* using the pipeline() function. They must manually: 1) Load the AutoTokenizer and AutoModel for a sentiment task, 2) Tokenize a sentence (convert it to [span_371](start_span)[span_371](end_span)input_ids and attention_mask[span_340](start_span)[span_340](end_span)), 3) Pass these tensors to the model, 4) Receive the output "logits," and 5) Apply a softmax function to interpret the logits as a human-readable prediction. This is a *critical* step in de-mystifying what these models actually do.

Week 21: Embeddings & Semantic Search

- **Topics:** What *are* embeddings? They are the *output* of models (like the ones from Week 20). An embedding is a high-dimensional vector (connecting back to Linear Algebra, Week 4) that numerically represents the *meaning* or *semantic content* of a piece of data (text, image, etc.). The learner will use a specific HF sentence-transformer model (e.g., all-MiniLM-L6-v2) to turn text into vectors. They will also learn to use Cosine Similarity (the mathematical formula) to measure the "closeness" of two vectors.
- **Weekly Project: Semantic Search Engine (Local).** The learner will: 1) Create a list of 50 example sentences, 2) Use a sentence-transformer model to embed all 50 sentences, storing them in a list (or NumPy array), 3) Write a function that takes a new "query" sentence, 4) Embeds the query, and 5) Uses a for loop to calculate the cosine similarity between the query vector and all 50 stored vectors. The function will then return the top 3 "most similar" sentences. This is a "brute-force" semantic search engine.

Week 22: Vector Databases

- **Topics:** A critical question: Why does the Week 21 project fail if the list contains 1 *billion* sentences? The "brute-force" similarity search is too slow. This necessitates a specialized database for high-speed vector search. The learner will be introduced to Vector Databases like ChromaDB (a fast, local-first option) and Pinecone (a powerful, cloud-based option).
- **Weekly Project: Scaling Semantic Search.** The learner will re-build the Week 21 project, but this time, they will *not* store the embeddings in a list. They will: 1) Instantiate a ChromaDB collection (or a Pinecone index), 2) Embed and store the 50 (or 500) sentences in the vector database, 3) Instead of a for loop, they will now use the database's built-in query() function. They will see that it returns the same (or similar) results, but it is infinitely more scalable.

Week 23: Retrieval-Augmented Generation (RAG - Part 1: Manual)

- **Topics:** The complete RAG workflow: **Retrieve, Augment, Generate**. RAG is the most common and effective pattern for "grounding" an LLM (i.e., giving it external knowledge) and preventing it from "hallucinating" (making things up).
- **Connecting the Dots:** RAG is not magic. It is a simple "recipe" that combines all the previous weeks:
    - **Retrieve:** Use a Vector Database (Week 22) to find relevant information.
    - **Augment:** Use Python string formatting (Week 1) to build a new, "augmented" prompt.
    - **Generate:** Send this prompt to an LLM via an API call (Week 6), like the OpenAI API.
- **Weekly Project: "Chat with a Document" (Manual).** The learner will: 1) Take a single text document (e.g., a short news article), 2) "Chunk" it into small, overlapping sentences or paragraphs , 3) Embed and store these chunks in their Vector DB , 4) Write a script that takes a user question, 5) Queries the DB to find the most *relevant* chunks, 6) *Manually* builds a prompt (e.g., f"Context: {retrieved_chunks}\n\nQuestion: {user_question}\n\nAnswer:"), and 7) Sends this "augmented" prompt to an LLM API (like

OpenAI's) and prints the response.

Week 24: RAG (Part 2: Frameworks)

- **Topics:** A reflection: Why was the Week 23 project so much manual work? (Chunking, embedding, querying, prompt-building...). This introduces the need for AI "orchestration frameworks" like LangChain or LlamaIndex, which automate this entire RAG pipeline.
- **Weekly Project: "Chat with a Document" (Automated).** The learner will re-build the *entire* Week 23 project. However, this time, they will do it in ~10-15 lines of code using LangChain. They will use a LangChain "loader" to read the document, a "text splitter" to chunk it, a "vector store" object (that wraps their DB), and a "retrieval chain" to do all the work. This project teaches the value of frameworks and how "real-world" AI applications are actually built. **This RAG-bot will be the second key asset for the deployment phase.**

Part 5: Phase 5 (Weeks 25-30): MLOps, Deployment & Capstone Project

A model that only runs in a Jupyter Notebook is a "hobby." A model that is accessible via a scalable API is a "product." This final phase is a single, 6-week capstone project. It covers the *critical* and *highly-sought-after* skills of MLOps (Machine Learning Operations): turning a model into a robust, scalable, and automated service that real users can access. This phase is the-end-to-end culmination of all previous learning.
Capstone Project Goal (Weeks 25-30): "End-to-End Deployed AI Service" The learner will choose *one* of their two primary assets to take to full production:

1. **Project A:** The Week 18 MNIST Classifier (a classical Deep Learning model).
2. **Project B:** The Week 24 RAG-bot (a modern Generative AI system).

Week 25 (The API):

- **Task:** Build a **FastAPI** web server. FastAPI is a modern, high-performance Python framework for building APIs. The learner will create a /predict endpoint that:
    - (For MNIST): Accepts an image file, loads the saved PyTorch model, performs inference, and returns a JSON response: {"digit": 7, "confidence": 0.98}.
    - (For RAG-bot): Accepts a JSON request {"question[span_415](start_span)[span_415](end_span)[span_417](start_span)[span_417](end_span)": "..."}, passes it to the LangChain RAG-bot, and returns a JSON response: {"answer": "..."}. This step effectively separates the "model logic" from the "application logic."

Week 26 (The Container):

- **Task:** Package the FastAPI application into a **Docker** container. The learner will write a Dockerfile that defines all the instructions to build a self-contained "box." This "box" (image) will bundle:
    1. A base Linux operating system.
    2. The Python interpreter.

3. All Python libraries from requirement[span_419](start_span)[span_419](end_span)[span_421](start_span)[span_421](end_span)[span_423](start_span)[span_423](end_span)s.txt.
4. The application code (the FastAPI server).
5. The model files (e.g., the .pt model file for MNIST or the ChromaDB data for the RAG-bot). The result is a portable, self-contained application that will run *identically* on any machine.

Week 27 (The Cloud):

- **Task:** Deploy the Docker container from Week 26 to a major cloud platform, making the AI service publicly accessible. The learner will choose *one* of the "Big 3" AI platforms to learn:
    - **Option 1 (AWS):** Deploy the container to **Amazon SageMaker**. This involves pushing the Docker image to Amazon ECR (Elastic Container Registry) and creating a SageMaker endpoint to serve it.
    - **Option 2 (GCP):** Deploy to **Google Cloud Vertex AI**. This involves pushing the image to the Artifact Registry and deploying it as a model on a Vertex AI Endpoint.
    - **Option 3 (Azure):** Deploy to **Azure Machine Learning**. This involves registering the model and deploying it as a "managed online endpoint".
    - **Result:** The AI service will now have a public URL. The learner can use a tool like curl or a Python script from *anywhere in the world* to call their API and get a prediction.

Week 28 (The Automation - CI/CD):

- **Task:** Automate the entire deployment process. The learner will create a **GitHub Actions** workflow. This is the core of Continuous Integration/Continuous Deployment (CI/CD) for MLOps. They will write a .yml file in their GitHub repository that triggers *automatically* every time they git push new code. This workflow will: 1. **CI (Integration):** Run automated tests (e.g., check that the API starts). 2. **CD (Deployment):** If tests pass, it will automatically build the new Docker image, push it to the cloud (e.g., AWS ECR), and deploy the new version to the cloud endpoint (e.g., SageMaker). This demonstrates an advanced, production-grade MLOps setup.

Week 29 (The Portfolio):

- **Task:** Create a "production-grade" **GitHub Portfolio**. An AI Engineer's portfolio is their most important asset. The learner will go back to their Capstone Project repository and write a comprehensive README.md file. This is not a "code dump"; it is a "sales page" for their skills. The README.md *must* include:
    1. **Project Title:** e.g., "End-to-End RAG Chatbot for".
    2. **Business Problem:** A clear 1-2 sentence description (e.g., "Answering user questions about a complex technical manual").
    3. **AI Solution:** A simple architecture diagram (which can be made in Mermaid) showing the flow: User -> API -> Docker -> RAG -> LLM -> Response.
    4. **Tech Stack:** A list of icons/badges (e.g., Python, FastAPI, Docker, AWS SageMaker, LangChain).

5. **Live API Endpoint:** The public URL from Week 27.
6. **How to Use:** A code snippet (using curl or Python requests) showing *exactly* how to call the live API.

Week 30 (The Interface):

- **Task:** (Optional but highly recommended) Build a simple web interface to "talk" to the deployed API. This provides a visual, interactive demo that is far more impressive in interviews than a code snippet.
    - **Option 1 (Easy):** Use **Streamlit**. Streamlit allows building beautiful data apps in pure Python. The learner can write a simple script that creates a text box, and when the user hits "Enter," it calls the deployed API (from Week 27) and displays the answer.
    - **Option 2 (Standard):** Build a basic **HTML/JavaScript** page. This page will have a text input and a button. The JavaScript's fetch() function will call the deployed API endpoint. This final step completes the "end-to-end" journey, from a blank Python file to a fully deployed, interactive AI application.

Part 7: Life as an AI Engineer: Daily Habits & Continuous Learning

This 7-month roadmap is designed to secure the *first* job. The following habits and strategies are essential for building a successful, long-term career in a field that evolves at an unprecedented pace.

Your New Day-to-Day: What Does an AI Engineer Actually Do?

A common misconception is that an AI Engineer's job is 100% "heads-down coding" of new models. The reality of a day-to-day role, especially in a team environment, is more varied and systems-oriented.

- **Morning :** The day often begins not with new code, but with analysis. This includes "analyzing training logs, comparing model performances" from overnight runs, "preparing summaries" for the team, and participating in "code reviews" (both giving and receiving feedback on colleagues' code).
- **Mid-day :** The core of the day is a mix of "heads-down coding" and "collaborative problem-solving." However, this is heavily interspersed with *meetings*. In large tech companies, this can be 50% or more of the day, including 1:1s, design reviews, and writing "design docs" and proposals.
- **Constant Task :** A primary job of an AI Engineer is "reviewing and interpreting output and telemetry data." Modern AI models, especially LLMs, are often "mysterious". A key, high-value skill is "picking apart apparently inexplicable results" to find the root cause of a failure, a bias, or a "hallucination."
- **The Real Job:** The job is not just "building models"; it's "systems-thinking". It involves "coaching" the AI with better data and prompts , analyzing its failures , and integrating it as a reliable, predictable component in a much larger software product.

How to Track Your Progress (Beyond "Percent Done")

During this 30-week plan, the learner must track progress like a professional AI project, not a simple "to-do" list. This means moving beyond "hours studied" and tracking "Key Performance Indicators" (KPIs) for each weekly project.

- **Metrics for Learning :** * **Model Accuracy / Performance:** Did the Week 18 MNIST model hit 98%? What was the final precision/recall for the Week 14 Spam Classifier?
    - **Training Time:** Did a change in the code make the model train faster?
    - **Deployment Milestones:** Binary "Done/Not Done" for key MLOps tasks. (e.g., "API built," "Dockerized," "Cloud Deployed," "CI/CD Pipeline Green").
- **Tools for Tracking :** Professionals use "Experiment Tracking" tools like MLflow or Neptune.ai. A beginner can start by creating a simple "Experiment Log" (e.g., a spreadsheet or a README.md in each project folder). For each project, this log should record:
    - **Hypothesis:** The goal (e.g., "A Random Forest will be more accurate than Logistic Regression for spam").
    - **Parameters:** The settings (e.g., "Random Forest with 100 trees, TfidfVectorizer").
    - **Results:** The "metrics" (e.g., "95% accuracy, 88% recall").
    - **Conclusion:** What was learned (e.g., "RF was more accurate but slower to train"). This instills the "scientific method" that is the foundation of all AI and machine learning.

Staying Current: The AI Field Changes Every 30 Days

This 7-month roadmap provides the *foundational skills* to get a first job. The *meta-skill* required to keep that job is the ability to learn and adapt continuously. The field is changing weekly, not yearly.

- **Daily:** Filter the noise. Follow a curated list of key researchers, engineers, and news sources on platforms like X (Twitter) and Reddit (e.g., r/learnmachinelearning, r/mlops, r/MachineLearning).
- **Weekly:** Read high-level summaries of new, important research. Sources like "ML-Papers-of-the-Week" digests are invaluable for spotting new trends (e.g., new "agentic" frameworks, new types of models, etc.).
- **Monthly:** "Practice the practice." Pick *one* new tool, paper, or technique that appeared in the weekly digests and spend a weekend replicating it with a mini-project. This "practice of learning" is what separates a good AI Engineer from a great one.

Appendix: The 30-Week AI Engineer Execution Plan

This table synthesizes the entire 7-month curriculum into a single, actionable checklist, providing the granular "day-to-day" and "weekly project" plan requested.

Week	Phase & Key Topic	Daily Tasks (Mon-Thurs)	Weekly Project (Fri-Sun)	Key Concepts to Master
W1	**Phase 1: Python Basics**	**Mon:** Setup (Python, VSCode). Run print("Hello	**Simple CLI Calculator:** Prompt user for 2	Variables, Data Types, Functions, Conditionals

Week	Phase & Key Topic	Daily Tasks (Mon-Thurs)	Weekly Project (Fri-Sun)	Key Concepts to Master
		World"). **Tue:** Learn variables (int, str, float), print(), f-strings. **Wed:** Learn functions (def), parameters, return values. **Thu:** Learn if/elif/else, comparison operators.	numbers and an operator. Return the result.	
W2	**Phase 1: Python Data Structures**	**Mon:** Master Lists (methods: .append(), .pop(), slicing). **Tue:** Master Dictionaries (keys, values, .get()). **Wed:** Master Sets (uniqueness) and Tuples (immutability). **Thu:** Learn for loops, while loops, range().	**Text-Based Hangman Game:** Use a List for words, a Set for guesses, and a while loop for the game.	Lists, Dictionaries, Sets, Tuples, Loops
W3	**Phase 1: Object-Oriented Programming (OOP)**	**Mon:** Read about OOP. Understand class, object, attribute, method. **Tue:** Learn the __init__() constructor (self). **Wed:** Build your first class (e.g., Dog with .bark() method). **Thu:** Learn inheritance (Parent/Child classes).	**Simple Banking App:** Create Customer class (name, PIN) and Account class (customer, balance, deposit(), withdraw()).	Class vs. Object, __init__, self, Attribute, Method
W4	**Phase 1: Algorithms & Linear Algebra Intuition**	**Mon:** Implement Bubble Sort to understand O(n^2). **Tue:** Learn what a Vector, Matrix, and Tensor	**Visualizing Linear Algebra:** Plot a vector. Define a 2x2 matrix. Multiply vector by matrix.	Algorithm Efficiency, Vector, Matrix, Tensor, Linear Transformation

Week	Phase & Key Topic	Daily Tasks (Mon-Thurs)	Weekly Project (Fri-Sun)	Key Concepts to Master
		are. **Wed:** Read: How data (features, datasets) is *represented* by matrices. **Thu:** Install Matplotlib. Learn to plot a 2D vector.	Plot the *new* vector.	
W5	**Phase 1: Calculus & Statistics Intuition**	**Mon:** Read: What is a derivative (slope)? What is a gradient? **Tue:** Read: What is the Chain Rule (at a high level)? **Wed:** Learn Stats: Mean, Median, Mode (centrality). **Thu:** Learn Stats: Variance, Standard Deviation (spread).	**Gradient Descent from Scratch:** Code y = x^2. Use its derivative (2x) to "walk" from a random x down to the minimum at x=0.	Gradient, Derivative, Gradient Descent, Mean, Variance
W6	**Phase 1: Files & Public APIs**	**Mon:** Learn to open(), read(), and write() .txt files. **Tue:** Learn to read/write .csv files with the csv module. **Wed:** Install requests. Learn GET requests. **Thu:** Learn what JSON is and how to parse it (it's just a Python dictionary).	**Public API "Weather App":** Prompt user for a city. Call a free weather API. Parse the JSON response. Print the forecast.	File I/O, API, requests, JSON
W7	**Phase 2: NumPy**	**Mon:** Install NumPy. Understand the ndarray. **Tue:** Master ndarray indexing, slicing, and filtering. **Wed:** Understand	**Image Manipulation with NumPy:** Load an image. Use array operations to change brightness, contrast, and crop it.	ndarray, Vectorization, Slicing, Broadcasting

Week	Phase & Key Topic	Daily Tasks (Mon-Thurs)	Weekly Project (Fri-Sun)	Key Concepts to Master
		Vectorization (NumPy vs. for loops). **Thu:** Learn broadcasting and universal functions (np.mean(), np.sum()).		
W8	**Phase 2: Pandas (Cleaning)**	**Mon:** Install Pandas. Learn Series and DataFrame. **Tue:** Master pd.read_csv(). Inspect data with .head(), .info(), .describe(). **Wed:** Find missing data (.isnull()). Handle it (.dropna(), .fillna()). **Thu:** Clean data (.astype(), .str.replace(), .drop_duplicates()).	**Real-World Data Cleaning:** Take a messy dataset. Clean missing values, types, and duplicates. Save the clean.csv.	DataFrame, Series, read_csv, fillna, dropna
W9	**Phase 2: Pandas (Analysis)**	**Mon:** Master filtering (loc, iloc, boolean indexing). **Tue:** Master grouping (.groupby()). **Wed:** Master aggregations (.mean(), .sum(), .count(), .value_counts()). **Thu:** Learn to merge() and join() two DataFrames.	**EDA on Telecom Churn:** Load churn data. Use .groupby() and aggregations to answer 3-4 business questions.	loc/iloc, groupby, Aggregation, merge
W10	**Phase 2: Matplotlib & Seaborn**	**Mon:** Install Matplotlib. Make a simple .plot() and .bar() chart. **Tue:** Customize plots (titles, labels,	**EDA Visualization Dashboard:** In a Jupyter Notebook, create 4-5 plots (hist, bar,	plt.plot(), plt.hist(), sns.heatmap(), sns.boxplot()

Week	Phase & Key Topic	Daily Tasks (Mon-Thurs)	Weekly Project (Fri-Sun)	Key Concepts to Master
		legends). **Wed:** Install Seaborn. Make a .boxplot() and .heatmap(). **Thu:** Understand *when* to use each plot (hist, bar, scatter, box).	heatmap) to visualize your findings from Week 9.	
W11	**Phase 2: Scikit-Learn (First Model)**	**Mon:** Install scikit-learn. Learn the .fit() / .predict() API. **Tue:** Understand Supervised Learning. Learn train_test_split. **Wed:** Learn Linear Regression (for numbers). **Thu:** Learn Logistic Regression (for categories).	**House Price Prediction (Regression):** Use Linear Regression to predict house prices. Evaluate with Mean Squared Error (MSE).	fit, predict, train_test_split, Linear Regression, MSE
W12	**Phase 2: Scikit-Learn (Evaluation)**	**Mon:** Learn *why* "Accuracy" can be misleading. **Tue:** Master the Confusion Matrix. **Wed:** Define and calculate: Accuracy, Precision, Recall, F1-score. **Thu:** Learn what Cross-Validation is and how to use cross_val_score.	**Breast Cancer Classifier (Logistic Regression):** Build a classifier. Report Accuracy, Precision, and Recall. Re-run with cross-validation.	Accuracy, Precision, Recall, Confusion Matrix, Cross-Validation
W13	**Phase 3: Supervised Learning (Trees)**	**Mon:** Read about Decision Trees (how they learn if/else rules). **Tue:** Learn TfidfVectorizer (converts text to numbers). **Wed:** Read about	**Spam Classifier (Part 1):** Use TfidfVectorizer and RandomForestClassifier to build a spam detector. Evaluate it.	Decision Tree, Random Forest, TfidfVectorizer, Ensemble

Week	Phase & Key Topic	Daily Tasks (Mon-Thurs)	Weekly Project (Fri-Sun)	Key Concepts to Master
		Random Forests (Ensemble "voting"). **Thu:** Train a RandomForestClassifier.		
W14	**Phase 3: Supervised Learning (Other)**	**Mon:** Learn Support Vector Machines (SVMs) (geometric intuition). **Tue:** Learn K-Nearest Neighbors (KNN) (voting intuition). **Wed:** Learn Naive Bayes (probabilistic intuition). **Thu:** Practice selecting models for different problems.	**Spam Classifier (Part 2):** Re-build the spam classifier with MultinomialNB and LinearSVC. Create a table comparing all 3 models.	SVM, KNN, Naive Bayes, Model Selection
W15	**Phase 3: Unsupervised Learning (Clustering)**	**Mon:** Understand Unsupervised vs. Supervised learning. **Tue:** Learn the K-Means Clustering algorithm (centroids, iteration). **Wed:** Learn the "Elbow Method" to find the best k. **Thu:** Train a KMeans model in Scikit-learn.	**Customer Segmentation:** Use K-Means on the "Mall Customer" dataset. Use the Elbow Method to pick k. Analyze the resulting clusters.	Unsupervised Learning, K-Means, Clustering, Elbow Method
W16	**Phase 3: Unsupervised Learning (Dim. Reduction)**	**Mon:** Read about the "Curse of Dimensionality." **Tue:** Learn what PCA (Principal Component Analysis) does. **Wed:** Connect PCA to Week 4	**Visualizing Customer Segments:** Take the Week 15 data. Use PCA to reduce it to 2D. Make a scatter plot, coloring by cluster ID.	PCA, Dimensionality Reduction, Eigenvectors

Week	Phase & Key Topic	Daily Tasks (Mon-Thurs)	Weekly Project (Fri-Sun)	Key Concepts to Master
		(Eigenvectors = "principal components"). **Thu:** Train a PCA model in Scikit-learn. Set n_components=2.		
W17	**Phase 3: Deep Learning (Intro & ANNs)**	**Mon:** Install PyTorch. Read PyTorch vs. TensorFlow. **Tue:** Learn the nn.Module, nn.Linear, Activation Functions (ReLU). **Wed:** Learn the PyTorch training loop (loss, optimizer, backprop). **Thu:** Build a simple ANN for the Iris dataset.	**Iris Classifier with an ANN:** Build and train your first neural network from scratch in PyTorch to classify the Iris dataset.	PyTorch, nn.Module, ANN/MLP, Activation Function, Training Loop
W18	**Phase 3: Deep Learning (CNNs)**	**Mon:** Read *why* ANNs fail on images. **Tue:** Learn the nn.Conv2d (Convolution) layer (filters). **Wed:** Learn the nn.MaxPool2d (Pooling) layer (downsampling). **Thu:** Build a simple CNN architecture (Conv -> Pool -> Conv -> Pool -> FC).	**Handwritten Digit Recognition (MNIST):** Build a CNN in PyTorch to classify MNIST digits. Aim for >98% accuracy. **Save this model!**	CNN, Convolution Layer, Pooling Layer, MNIST
W19	**Phase 4: Hugging Face (The Easy Way)**	**Mon:** Create a Hugging Face (HF) account. **Tue:** Read about the Transformer	**One-Line AI Tools:** Build a script that uses the HF pipeline() to perform 3 tasks:	Hugging Face, Transformers, pipeline()

Week	Phase & Key Topic	Daily Tasks (Mon-Thurs)	Weekly Project (Fri-Sun)	Key Concepts to Master
		architecture (high-level). **Wed:** Install transformers. Learn the pipeline() function. **Thu:** Use pipeline() for sentiment analysis and text summarization.	Sentiment Analysis, Text Summarization, and Image Captioning.	
W20	**Phase 4: Hugging Face (The "Real" Way)**	**Mon:** Learn what a Tokenizer is. **Tue:** Load a pre-trained AutoTokenizer. **Wed:** Load a pre-trained AutoModel. **Thu:** Understand how to feed tokens (input_ids, attention_mask) to a model and interpret the output "logits".	**Re-Build the Sentiment Analyzer:** Replicate the Week 19 project *without* pipeline(). Manually load the tokenizer and model.	AutoTokenizer, AutoModel, Tokens, Logits, input_ids
W21	**Phase 4: Embeddings & Semantic Search**	**Mon:** Learn what an "Embedding" is (a vector of meaning). **Tue:** Install sentence-transformers. Load a model. **Wed:** Learn Cosine Similarity (the math for "closeness"). **Thu:** Write a Python function for cosine similarity.	**Semantic Search Engine (Local):** Embed 50 sentences. Write a function that takes a query, embeds it, and uses a for loop to find the most similar sentence.	Embeddings, Semantic Search, Cosine Similarity
W22	**Phase 4: Vector Databases**	**Mon:** Read: Why does Week 21 fail at 1 billion sentences? **Tue:** Install chromadb.	**Scaling Semantic Search:** Re-build the Week 21 project. Store the embeddings in	Vector Database, ChromaDB, Pinecone, Scalable Search

Week	Phase & Key Topic	Daily Tasks (Mon-Thurs)	Weekly Project (Fri-Sun)	Key Concepts to Master
		Learn the basic API. **Wed:** Create a "collection." Add embeddings and text. **Thu:** Use the collection.query() function to find similar vectors.	ChromaDB. Use collection.query() instead of a for loop.	
W23	**Phase 4: RAG (Part 1: Manual)**	**Mon:** Learn the RAG workflow: Retrieve, Augment, Generate. **Tue:** Learn "Chunking" (splitting big docs). **Wed:** Get an OpenAI (or other LLM) API key. **Thu:** Practice manually creating a prompt with f-strings.	**"Chat with a Document" (Manual):** Chunk a doc, store in ChromaDB. Take a query, query() ChromaDB (Retrieve), build a prompt (Augment), send to OpenAI (Generate).	RAG, Retrieve, Augment, Generate, Chunking, Prompt Engineering
W24	**Phase 4: RAG (Part 2: Frameworks)**	**Mon:** Read: Why was Week 23 so much work? **Tue:** Install langchain. Read the high-level concepts. **Wed:** Learn LangChain's "Loaders," "Splitters," and "Vector Stores." **Thu:** Learn how to build a RetrievalQA chain.	**"Chat with a Document" (Automated):** Re-build the *entire* Week 23 project using a LangChain chain. Do it in ~10 lines of code.	LangChain, LlamaIndex, Orchestration, AI Pipelines
W25	**Phase 5: Capstone (The API)**	**Mon:** Install fastapi and uvicorn. **Tue:** Create a simple "hello world" API. **Wed:** Design the /predict endpoint (request/response models). **Thu:**	**Build FastAPI Server:** Create a main.py with a FastAPI app. Load your saved model. Create a /predict endpoint.	FastAPI, Uvicorn, API, Endpoint, JSON

Week	Phase & Key Topic	Daily Tasks (Mon-Thurs)	Weekly Project (Fri-Sun)	Key Concepts to Master
		Write the API logic to load your model (MNIST or RAG) and return a JSON prediction.		
W26	**Phase 5: Capstone (The Container)**	**Mon:** Install Docker. Run docker run hello-world. **Tue:** Learn Dockerfile syntax (FROM, COPY, RUN, CMD). **Wed:** Write a Dockerfile for your FastAPI app. **Thu:** Build the image (docker build) and run it (docker run). Test it.	**Dockerize the AI App:** Write a Dockerfile that packages your FastAPI app, requirements.txt, and model file into a runnable container.	Docker, Dockerfile, Container, Image, Port Mapping
W27	**Phase 5: Capstone (The Cloud)**	**Mon:** Create an account (AWS, GCP, or Azure). **Tue:** Learn the chosen platform's container registry (ECR, Artifact Registry, ACR). **Wed:** Learn the model serving tool (SageMaker, Vertex AI, Azure ML). **Thu:** Push your Docker image to the registry.	**Deploy to the Cloud:** Deploy your Docker image to a managed cloud endpoint (e.g., SageMaker or Vertex AI). Get a public API URL.	AWS SageMaker, GCP Vertex AI, Azure ML, Cloud Deployment
W28	**Phase 5: Capstone (The Automation)**	**Mon:** Read about CI/CD. **Tue:** Learn GitHub Actions basics (workflows, jobs, steps). **Wed:** Find a "Deploy to SageMaker/Vertex " GitHub Action. **Thu:** Create the .yml file in	**Create CI/CD Pipeline:** Build a GitHub Action that *automatically* (on git push) builds your Docker image and deploys it to the cloud endpoint.	CI/CD, GitHub Actions, MLOps, Automation

Week	Phase & Key Topic	Daily Tasks (Mon-Thurs)	Weekly Project (Fri-Sun)	Key Concepts to Master
		.github/workflows/.		
W29	**Phase 5: Capstone (The Portfolio)**	**Mon:** Read about good GitHub README.md files. **Tue:** Create an architecture diagram for your project. **Wed:** Write the "Business Problem" and "AI Solution" sections. **Thu:** Add the "Tech Stack" and "How to Use" (with code!) sections.	**Build GitHub Portfolio:** Write a "sales page" README.md for your capstone project. Include the live API URL and code examples.	Portfolio, README.md, Documentation
W30	**Phase 5: Capstone (The Interface)**	**Mon:** Install streamlit. **Tue:** Create a "hello world" Streamlit app. **Wed:** Add a text input box and a button. **Thu:** Write the requests code to call your *deployed cloud API* from the Streamlit app.	**Build a Streamlit UI:** Create a simple web app that provides a UI for your deployed model. Host it on Streamlit Community Cloud.	Streamlit, Frontend, UI, Full-Stack AI

Works cited

1. How long to realistically become good at AI/ML if I study 8 hrs/day and focus on building real-world projects? : r/learnmachinelearning - Reddit, https://www.reddit.com/r/learnmachinelearning/comments/1ngq9xu/how_long_to_realistically_become_good_at_aiml_if/ 2. From Zero to AI Engineer in Less Than 6 Months!, https://zerotomastery.io/blog/ai-engineer-in-just-6-months/ 3. Roadmap to Becoming an AI Engineer in 8 to 12 Months (From Scratch). - Reddit, https://www.reddit.com/r/learnmachinelearning/comments/1g6d4cz/roadmap_to_becoming_an_ai_engineer_in_8_to_12/ 4. 6-Month AI Engineer Roadmap - OpenCV, https://opencv.org/blog/ai-engineer-roadmap/ 5. AI Engineering: A *Realistic* Roadmap for Beginners - YouTube, https://www.youtube.com/watch?v=dbUIjFXIpis 6. “Your Ultimate AI Roadmap: From Beginner to Expert in 6 Months” | by AI Tech Daily, https://medium.com/@aitechdaily/your-ultimate-ai-roadmap-from-beginner-to-expert-in-6-months-12d2af4efd04 7. How to Learn AI From Scratch in 2025: A Complete Guide From the Experts - DataCamp, https://www.datacamp.com/blog/how-to-learn-ai 8. Gen AI Engineer vs ML Engineer 🤖| Skills, Salary & Future Scope, https://www.youtube.com/watch?v=T6OLZ6Fg74w

9. AI Engineer vs ML Engineer: Demand, Salaries, and Career Growth in 2025 - Vettio, https://vettio.com/blog/ai-engineer-vs-ml-engineer/ 10. AI Engineer vs. ML Engineer: Roles, Skills, and Career Paths - ARTiBA, https://www.artiba.org/blog/ai-engineer-vs-ml-engineer-roles-skills-and-career-paths 11. AI Engineer - Developer Roadmaps, https://roadmap.sh/ai-engineer 12. 33 Machine Learning Projects for All Levels in 2025 - DataCamp, https://www.datacamp.com/blog/machine-learning-projects-for-all-levels 13. AI Python for Beginners - DeepLearning.AI, https://www.deeplearning.ai/short-courses/ai-python-for-beginners/ 14. Python for Data Science course - Cognitive Class, https://cognitiveclass.ai/courses/python-for-data-science 15. Python for Data Science, AI & Development - Coursera, https://www.coursera.org/learn/python-for-applied-data-science-ai 16. Object-Oriented Programming (OOP) in Python, https://realpython.com/python3-object-oriented-programming/ 17. Struggling with Python OOP—Seeking Advice Before Diving Into AI : r/learnpython - Reddit, https://www.reddit.com/r/learnpython/comments/1jcnj6o/struggling_with_python_oopseeking_advice_before/ 18. What Math Do I Need to Know for AI?: 5 Types to Know | Coursera, https://www.coursera.org/articles/what-math-do-i-need-to-know-for-ai 19. Maths for Machine Learning - GeeksforGeeks, https://www.geeksforgeeks.org/machine-learning/machine-learning-mathematics/ 20. The Essential Math You Need for AI and Machine Learning (With Roadmap and Resources) | by Pravin More | Medium, https://medium.com/@morepravin1989/the-essential-math-you-need-for-ai-and-machine-learning-with-roadmap-and-resources-0a7d332466bb 21. Python Fundamentals for Data Engineers: Loops Cheatsheet | Codecademy, https://www.codecademy.com/learn/decp-python-fundamentals-for-data-engineers/modules/learn-python3-loops/cheatsheet 22. Lists, Tuples, Dictionaries, and Sets · HonKit - Computer Science Department, https://cs.du.edu/~intropython/byte-of-python/data_structures.html 23. 5. Data Structures — Python 3.14.0 documentation, https://docs.python.org/3/tutorial/datastructures.html 24. How to Choose the Right Data Structure in Python: Lists, Tuples, Sets, or Dicts?, https://www.mygreatlearning.com/blog/choose-right-python-data-structure/ 25. Python for AI: Week 3 — Lists, Tuples, Sets, and Dictionaries | by Ebrahim Mousavi | Medium, https://medium.com/@ebimsv/mastering-python-for-ai-week-3-lists-tuples-sets-and-dictionaries-c606321d2d5a 26. Python Data Structures: Lists, Dictionaries, Sets, Tuples - Dataquest, https://www.dataquest.io/blog/data-structures-in-python/ 27. A Beginner's Guide to Python Object-Oriented Programming (OOP) - Kinsta, https://kinsta.com/blog/python-object-oriented-programming/ 28. Object Oriented Programming with Python - Full Course for Beginners - YouTube, https://www.youtube.com/watch?v=Ej_02ICOIgs 29. Sorting Algorithms in Python - GeeksforGeeks, https://www.geeksforgeeks.org/python/sorting-algorithms-in-python/ 30. Sorting and Searching Algorithms in Python: Optimizing Data Management - 4Geeks, https://4geeks.com/lesson/sorting-and-search-algorithms-in-python 31. Sorting Algorithms in Python, https://realpython.com/sorting-algorithms-python/ 32. Complete Guide on Sorting Techniques in Python [2025 Edition] - Analytics Vidhya, https://www.analyticsvidhya.com/blog/2024/01/sorting-techniques-in-python/ 33. What Is Linear Algebra for Machine Learning? - IBM, https://www.ibm.com/think/topics/linear-algebra-for-machine-learning 34. Linear Algebra Operations For Machine Learning - GeeksforGeeks,

https://www.geeksforgeeks.org/machine-learning/ml-linear-algebra-operations/ 35. Key Linear Algebra Concepts Every Machine Learning Engineer Should Master, https://datascienceafrica.medium.com/key-linear-algebra-concepts-every-machine-learning-engineer-should-master-9103a76e846e 36. Eigenvectors and Eigenvalues - Detailed Explanation on Eigenvectors and Eigenvalues - Machine Learning Plus, https://www.machinelearningplus.com/linear-algebra/eigenvectors-and-eigenvalues/ 37. Backpropagation - Wikipedia, https://en.wikipedia.org/wiki/Backpropagation 38. Backpropagation in Deep Learning: The Key to Optimizing Neural Networks - Medium, https://medium.com/@juanc.olamendy/backpropagation-in-deep-learning-the-key-to-optimizing-neural-networks-7c063a03f677 39. Backpropagation – The Math Behind Optimization - 365 Data Science, https://365datascience.com/trending/backpropagation/ 40. Probability & Statistics for Machine Learning & Data Science | Coursera, https://www.coursera.org/learn/machine-learning-probability-and-statistics 41. Mastering the Basics Part 12: Understanding of Statistics and Probability for Data and AI | by Vaishali Lambe - The Curious Mind | Medium, https://medium.com/@curiousmind1786/mastering-the-basics-part-12-understanding-of-statistics-and-probability-for-data-and-ai-31472eaff9bc 42. Statistics and Probability Concepts for Data Science - Analytics Vidhya, https://www.analyticsvidhya.com/blog/2021/04/statistics-and-probability-concepts-for-data-science/ 43. Complete Statistics For Data Science In 6 hours By Krish Naik - YouTube, https://www.youtube.com/watch?v=LZzq1zSL1bs 44. A Data Scientist's Guide to Gradient Descent and Backpropagation Algorithms, https://developer.nvidia.com/blog/a-data-scientists-guide-to-gradient-descent-and-backpropagation-algorithms/ 45. Training for AI engineers | Microsoft Learn, https://learn.microsoft.com/en-us/training/career-paths/ai-engineer 46. How to Combine Pandas, NumPy, and Scikit-learn Seamlessly - MachineLearningMastery.com, https://machinelearningmastery.com/how-to-combine-pandas-numpy-and-scikit-learn-seamlessly/ 47. Learn - NumPy, https://numpy.org/learn/ 48. Python for Data Science - Course for Beginners (Learn Python, Pandas, NumPy, Matplotlib), https://www.youtube.com/watch?v=LHBE6Q9XlzI 49. scikit-learn: machine learning in Python — scikit-learn 1.7.2 documentation, https://scikit-learn.org/ 50. Getting Started — scikit-learn 1.7.2 documentation, https://scikit-learn.org/stable/getting_started.html 51. Pythonic Data Cleaning With pandas and NumPy, https://realpython.com/python-data-cleaning-numpy-pandas/ 52. Basic Steps When Cleaning a Data Set Using Pandas | by Will Newton - Medium, https://medium.com/the-innovation/basic-steps-when-cleaning-a-data-set-using-pandas-3576e716173d 53. 12 free Data Science projects to practice Python and Pandas - DataWars, https://www.datawars.io/articles/12-free-data-science-projects-to-practice-python-and-pandas 54. Data Cleaning Project Walk-through – Dataquest, https://www.dataquest.io/tutorial/data-cleaning-project-walk-through/ 55. Data Cleaning in Pandas | Python Pandas Tutorials - YouTube, https://www.youtube.com/watch?v=bDhvCp3_lYw 56. Exploratory Data Analysis (EDA) with NumPy, Pandas, Matplotlib and Seaborn, https://www.geeksforgeeks.org/data-analysis/eda-with-NumPy-Pandas-Matplotlib-Seaborn/ 57. Topic 1. Exploratory Data Analysis with Pandas - Kaggle, https://www.kaggle.com/code/kashnitsky/topic-1-exploratory-data-analysis-with-pandas 58. Anas1108/Exploratory-Data-Analysis-using-Pandas-and-Matplotlib - GitHub, https://github.com/Anas1108/Exploratory-Data-Analysis-using-Pandas-and-Matplotlib 59. Exploratory Data Analysis (EDA) with Python and Pandas | by Durga Gadiraju - Medium, https://medium.com/itversity/exploratory-data-analysis-eda-with-python-and-pandas-fc441616ffd

c 60. Mastering Logistic Regression with Scikit-Learn: A Complete Guide | DigitalOcean, https://www.digitalocean.com/community/tutorials/logistic-regression-with-scikit-learn 61. Python Logistic Regression Tutorial with Sklearn & Scikit - DataCamp, https://www.datacamp.com/tutorial/understanding-logistic-regression-python 62. Learning Model Building in Scikit-learn - GeeksforGeeks, https://www.geeksforgeeks.org/machine-learning/learning-model-building-scikit-learn-python-machine-learning-library/ 63. 15+ Pandas Project Ideas for Beginners in 2025 - GeeksforGeeks, https://www.geeksforgeeks.org/blogs/pandas-project-ideas/ 64. Random Forest Classification in Python With Scikit-Learn: Step-by-Step Guide (with Code Examples) | DataCamp, https://www.datacamp.com/tutorial/random-forests-classifier-python 65. Supervised Machine Learning - GeeksforGeeks, https://www.geeksforgeeks.org/machine-learning/supervised-machine-learning/ 66. 1.10. Decision Trees — scikit-learn 1.7.2 documentation, https://scikit-learn.org/stable/modules/tree.html 67. 1.4. Support Vector Machines - Scikit-learn, https://scikit-learn.org/stable/modules/svm.html 68. ANN vs CNN vs RNN: Understanding the Difference - Codoid, https://codoid.com/ai/ann-vs-cnn-vs-rnn-understanding-the-difference/ 69. Deep Learning Tutorial - GeeksforGeeks, https://www.geeksforgeeks.org/deep-learning/deep-learning-tutorial/ 70. ANN vs. RNN vs. CNN: Decoding the Deep Learning Alphabet Soup | atalupadhyay, https://atalupadhyay.wordpress.com/2025/03/01/ann-vs-rnn-vs-cnn-decoding-the-deep-learning-alphabet-soup/ 71. Building Blocks of Deep Learning: ANN, CNN, RNN, and Transformers | by Keerthanams, https://medium.com/@keerthanams1208/building-blocks-of-deep-learning-ann-cnn-rnn-and-transformers-0e0f830d090f 72. 12 Types of Neural Networks in Deep Learning - Analytics Vidhya, https://www.analyticsvidhya.com/blog/2020/02/cnn-vs-rnn-vs-mlp-analyzing-3-types-of-neural-networks-in-deep-learning/ 73. A Beginner's Guide to Building a Spam Classifier with Python and Scikit-learn - Medium, https://medium.com/@thestatisticsassignmenth/a-beginners-guide-to-building-a-spam-classifier-with-python-and-scikit-learn-253663604e02 74. 100+ Machine Learning Projects with Source Code [2025] - GeeksforGeeks, https://www.geeksforgeeks.org/machine-learning/machine-learning-projects/ 75. adhikareeprayush/Spam-Detection: A machine learning model to classify emails as spam or not spam using a dataset of email content. - GitHub, https://github.com/adhikareeprayush/Spam-Detection 76. Topic 7. Unsupervised learning: PCA and clustering - Kaggle, https://www.kaggle.com/code/kashnitsky/topic-7-unsupervised-learning-pca-and-clustering 77. 2.3. Clustering — scikit-learn 1.7.2 documentation, https://scikit-learn.org/stable/modules/clustering.html 78. K-Means Clustering Made Easy: Step-by-Step Guide with Scikit-Learn Python Code, https://www.youtube.com/watch?v=hBHoEbZohI0 79. How to build a segmentation with k-means clustering and PCA in Python with sklearn, https://www.stepbystepdatascience.com/k-means-and-pca-in-python-with-sklearn 80. Customer Segmentation using K-Means Algorithm in Python | Medium, https://medium.com/@ugursavci/step-by-step-customer-segmentation-using-k-means-and-pca-in-python-5733822295b6 81. Customer Segmentation using Unsupervised Machine Learning in Python - GeeksforGeeks, https://www.geeksforgeeks.org/machine-learning/customer-segmentation-using-unsupervised-machine-learning-in-python/ 82. Project Tutorial: Customer Segmentation Using K-Means

Clustering - Dataquest,
https://www.dataquest.io/blog/customer-segmentation-using-k-means-clustering/ 83. Scikit Learn - Dimensionality Reduction using PCA - Tutorials Point,
https://www.tutorialspoint.com/scikit_learn/scikit_learn_dimensionality_reduction_using_pca.htm 84. Dimensionality Reduction with Scikit-Learn | by Dr. Deepak Kumar Singh | Medium,
https://medium.com/@deepak.engg.phd/dimensionality-reduction-with-scikit-learn-ee5d2b69225b 85. PCA — scikit-learn 1.7.2 documentation,
https://scikit-learn.org/stable/modules/generated/sklearn.decomposition.PCA.html 86. 7.5. Unsupervised dimensionality reduction - Scikit-learn,
https://scikit-learn.org/stable/modules/unsupervised_reduction.html 87. Linear Algebra Essentials for Machine Learning Developers - FullStack Labs,
https://www.fullstack.com/labs/resources/blog/linear-algebra-essentials-for-machine-learning-developers 88. How to Combine PCA & K-Means Clustering in Python - 365 Data Science,
https://365datascience.com/tutorials/python-tutorials/pca-k-means/ 89. Dimensionality Reduction with Neighborhood Components Analysis - Scikit-learn,
https://scikit-learn.org/stable/auto_examples/neighbors/plot_nca_dim_reduction.html 90. PyTorch Vs TensorFlow (2025): An In-Depth Comparison - AceCloud,
https://acecloud.ai/blog/pytorch-vs-tensorflow/ 91. Is PyTorch better than TensorFlow for beginners in AI/ML in 2025?,
https://dev-discuss.pytorch.org/t/is-pytorch-better-than-tensorflow-for-beginners-in-ai-ml-in-2025/3033 92. TensorFlow vs PyTorch: Which Framework Should You Learn in 2025? | Udacity,
https://www.udacity.com/blog/2025/06/tensorflow-vs-pytorch-which-framework-should-you-learn-in-2025.html 93. TensorFlow or PyTorch: which to choose in 2025? : r/learnmachinelearning - Reddit,
https://www.reddit.com/r/learnmachinelearning/comments/1hu9mkt/tensorflow_or_pytorch_which_to_choose_in_2025/ 94. Introduction - Hugging Face LLM Course,
https://huggingface.co/learn/llm-course/en/chapter2/1 95. 30 Artificial Intelligence Project Ideas in 2026 [Trending] - Simplilearn.com,
https://www.simplilearn.com/tutorials/artificial-intelligence-tutorial/ai-project-ideas 96. Learn How to Use Chroma DB: A Step-by-Step Guide | DataCamp,
https://www.datacamp.com/tutorial/chromadb-tutorial-step-by-step-guide 97. RAG Tutorial: A Beginner's Guide to Retrieval Augmented Generation - SingleStore,
https://www.singlestore.com/blog/a-guide-to-retrieval-augmented-generation-rag/ 98. How do Transformers work? - Hugging Face LLM Course,
https://huggingface.co/learn/llm-course/en/chapter1/4 99. Total noob's intro to Hugging Face Transformers, https://huggingface.co/blog/noob_intro_transformers 100. Getting Started With Hugging Face in 10 Minutes, https://huggingface.co/blog/proflead/hugging-face-tutorial 101. RAG Explained For Beginners - YouTube, https://www.youtube.com/watch?v=_HQ2H_0Ayy0 102. Complete Tutorial on Vector Database - Learn ChromaDB, Pinecone & Weaviate - YouTube, https://www.youtube.com/watch?v=8KrTO9bS91s 103. What is a Vector Database & How Does it Work? Use Cases + Examples - Pinecone,
https://www.pinecone.io/learn/vector-database/ 104. Vector Database Tutorial FOR Beginners with PineCone [Hands on Lab] - YouTube, https://www.youtube.com/watch?v=3OGWzDNMeaQ 105. Understanding Retrieval Augmented Generation (RAG): a quick tutorial | by Mallika Dey,
https://medium.com/@dey.mallika/understanding-retrieval-augmented-generation-rag-a-quick-tutorial-d9710c005cbe 106. Retrieval Augmented Generation (RAG) from Scratch — Tutorial For Dummies,
https://dev.to/zachary62/retrieval-augmented-generation-rag-from-scratch-tutorial-for-dummies-5

08a 107. Retrieval Augmented Generation (RAG) and Vector Databases (Part 15 of 18) | Microsoft Learn, https://learn.microsoft.com/en-us/shows/generative-ai-for-beginners/retrieval-augmented-generation-rag-and-vector-databases-generative-ai-for-beginners 108. ML Project Ideas? : r/learnmachinelearning - Reddit, https://www.reddit.com/r/learnmachinelearning/comments/1bqc3dw/ml_project_ideas/ 109. Capstone Projects - FourthBrain, https://fourthbrain.ai/capstones/ 110. GURPREETKAURJETHRA/END-TO-END-GENERATIVE-AI-PROJECTS - GitHub, https://github.com/GURPREETKAURJETHRA/END-TO-END-GENERATIVE-AI-PROJECTS 111. Step-by-Step Guide to Deploying Machine Learning Models with FastAPI and Docker, https://machinelearningmastery.com/step-by-step-guide-to-deploying-machine-learning-models-with-fastapi-and-docker/ 112. How to Deploy Machine Learning Models with FastAPI, Docker, and Fly.io - YouTube, https://www.youtube.com/watch?v=jzGzw98Eikk 113. Deploying a Machine Learning Model with FastAPI and Docker — A Step-by-Step Guide, https://python.plainenglish.io/deploying-a-machine-learning-model-with-fastapi-and-docker-a-step-by-step-guide-c07abc9c1afe 114. Serve a machine learning model using Sklearn, FastAPI, and Docker. - Medium, https://medium.com/analytics-vidhya/serve-a-machine-learning-model-using-sklearn-fastapi-and-docker-85aabf96729b 115. Serve your first model with Scikit-Learn + Flask + Docker | by Carl W. Handlin | Rappi Tech, https://engineering.rappi.com/serve-your-first-model-with-scikit-learn-flask-docker-df95efbbd35e 116. Implement MLOps - Amazon SageMaker AI - AWS Documentation, https://docs.aws.amazon.com/sagemaker/latest/dg/mlops.html 117. Getting Started with Machine Learning on Amazon SageMaker AI - AWS, https://aws.amazon.com/sagemaker/ai/getting-started/ 118. MLOps with Amazon SageMaker - AWS, https://aws.amazon.com/awstv/watch/5057107a7fc/ 119. A Practical Guide to MLOps in AWS Sagemaker — Part I | by Akash Agnihotri - Medium, https://medium.com/data-science/a-practical-guide-to-mlops-in-aws-sagemaker-part-i-1d28003f565 120. Getting started with MLOps on Amazon SageMaker for generative AI - YouTube, https://www.youtube.com/watch?v=EjiYnHkmn1I 121. Vertex AI Platform | Google Cloud, https://cloud.google.com/vertex-ai 122. AutoML beginner's guide | Vertex AI - Google Cloud Documentation, https://docs.cloud.google.com/vertex-ai/docs/beginner/beginners-guide 123. Deploying Machine Learning models w/ Vertex AI on GCP - Supertype, https://supertype.ai/notes/deploying-machine-learning-models-with-vertex-ai-on-google-cloud-platform 124. How to Build & Deploy Machine Learning Models with Vertex AI | Aionlinecourse, https://www.aionlinecourse.com/blog/how-to-build-deploy-machine-learning-models-with-vertex-ai 125. Deploy ML Models in a Production setting using GCP | by ylnhari - Medium, https://medium.com/applied-ml-mlops/register-deploy-ml-models-in-production-using-gcp-943c04254f1b 126. Tutorial: Deploy a model - Azure Machine Learning | Microsoft Learn, https://learn.microsoft.com/en-us/azure/machine-learning/tutorial-deploy-model?view=azureml-api-2 127. Deploy machine learning models to Azure - Microsoft Learn, https://learn.microsoft.com/en-us/Azure/machine-learning/how-to-deploy-and-where?view=azureml-api-1 128. Quickstart: Get started with Azure Machine Learning, https://learn.microsoft.com/en-us/azure/machine-learning/tutorial-azure-ml-in-a-day?view=azureml-api-2 129. Deploy and score a machine learning model by using an online endpoint, https://learn.microsoft.com/en-us/azure/machine-learning/how-to-deploy-online-endpoints?view=azureml-api-2 130. Build and deploy ML Models on Azure Machine Learning Studio - YouTube, https://www.youtube.com/watch?v=zB4NDi-fLvo 131. GitHub Actions for CI/CD - Azure Machine

Learning | Microsoft Learn,
https://learn.microsoft.com/en-us/azure/machine-learning/how-to-github-actions-machine-learning?view=azureml-api-2 132. Day 13 – Getting Started with GitHub Actions | CI/CD with GitHub Actions for Beginners, https://www.youtube.com/watch?v=8hwcRnP3agc 133. How to build a CI/CD pipeline with GitHub Actions in four simple steps,
https://github.blog/enterprise-software/ci-cd/build-ci-cd-pipeline-github-actions-four-steps/ 134. A Beginner's Guide to CI/CD for Machine Learning - DataCamp,
https://www.datacamp.com/tutorial/ci-cd-for-machine-learning 135. Streamlined CI/CD Pipelines Using Claude Code & GitHub Actions,
https://medium.com/@itsmybestview/streamlined-ci-cd-pipelines-using-claude-code-github-actions-74be17e51499 136. Creating Your Professional Portfolio on GitHub and Kaggle - The ML Mastermind,
https://themlmastermind.com/f/creating-your-professional-portfolio-on-github-and-kaggle 137. Efficient Way of Building Portfolio : r/learnmachinelearning - Reddit,
https://www.reddit.com/r/learnmachinelearning/comments/1jbp5t9/efficient_way_of_building_portfolio/ 138. How to Make a Data Science Portfolio With GitHub Pages (2025) - YouTube,
https://www.youtube.com/watch?v=D9CLhQdLp8w 139. The Ultimate Guide to Building a Machine Learning Portfolio That Lands Jobs,
https://machinelearningmastery.com/ultimate-guide-building-machine-learning-portfolio-lands-jobs/ 140. A Successful AI/ML Digital Portfolio, Example of a good Github Repo! - YouTube,
https://www.youtube.com/watch?v=gXtpwPHI84o 141. 40+ Artificial Intelligence Project Ideas for Beginners [2025] - ProjectPro,
https://www.projectpro.io/article/artificial-intelligence-project-ideas/461 142. Daily Life of a Machine Learning Engineer - Noble Desktop,
https://www.nobledesktop.com/careers/machine-learning-engineer/daily-life 143. Fellow ML/AI engineers, what does your daily work schedule look like? - Reddit,
https://www.reddit.com/r/MLQuestions/comments/1kblpth/fellow_mlai_engineers_what_does_your_daily_work/ 144. A Day in the Life of an AI Engineer - AI Degree Guide,
https://aidegreeguide.com/blog/a-day-in-the-life-of-an-ai-engineer/ 145. Stanford's Practical Guide to 10x Your AI Productivity | Jeremy Utley,
https://www.youtube.com/watch?v=yMOmmnjy3sE&vl=en 146. How to Become a Self Taught AI Engineer Today, https://www.youtube.com/watch?v=MlR1qlgosaU 147. Best Practices for Progress Tracking in AI-Driven Projects - Sidetool,
https://www.sidetool.co/post/best-practices-for-progress-tracking-in-ai-driven-projects/ 148. 13 Best Tools for ML Experiment Tracking and Management in 2025,
https://neptune.ai/blog/best-ml-experiment-tracking-tools 149. dair-ai/ML-Papers-of-the-Week - GitHub, https://github.com/dair-ai/ML-Papers-of-the-Week